IFM 3.1 (PO23) / IFM 5.21 (PO18): Compilerbau (Winter 2024/25)

Foo Bar

21. März 2026

Inhaltsverzeichnis

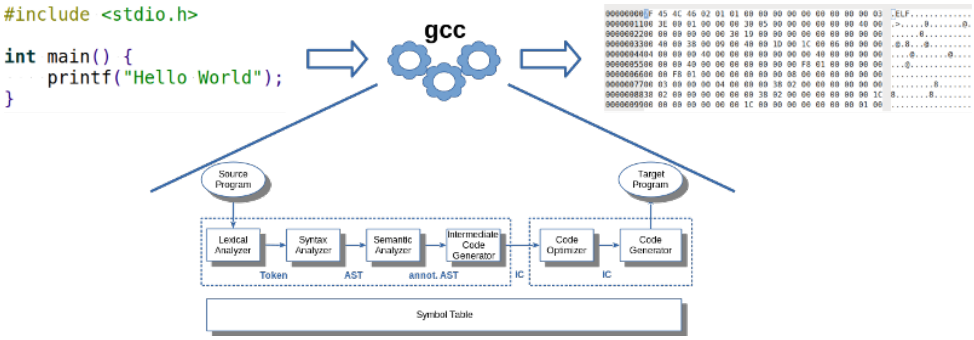
1 Syllabus **2**
- 1.1 Kursbeschreibung . . . 2
- 1.2 Überblick Modulinhalte . . . 2
- 1.3 Team . . . 3
- 1.4 Kursformat . . . 3
- 1.5 Fahrplan . . . 3
- 1.6 Prüfungsform, Note und Credits . . . 4
  - 1.6.1 Prüfung im ersten Zeitraum” . . . 4
  - 1.6.2 Prüfung im zweiten Zeitraum” . . . 5
- 1.7 Materialien . . . 5
- 1.8 Förderungen und Kooperationen . . . 5
  - 1.8.1 Kooperation mit University of Alberta, Edmonton (Kanada) . . . 5
- 1.9 LICENSE . . . 5

2 Vorlesungsunterlagen **6**
- 2.1 Syntaktische Analyse . . . 6
  - 2.1.1 CFG . . . 6
  - 2.1.2 Parser mit ANTLR generieren . . . 13
- 2.2 Test Markdown . . . 25
  - 2.2.1 Test Markdown . . . 27
  - 2.2.2 NN02 - Lineare Regression und Gradientenabstieg . . . 54
  - 2.2.3 subfolder . . . 57

3 Praktikum **58**
- 3.1 Blatt 04: Semantische Analyse . . . 58
  - 3.1.1 Zusammenfassung . . . 58
  - 3.1.2 Methodik . . . 58
  - 3.1.3 Sprachdefinition . . . 58
  - 3.1.4 Aufgaben . . . 64
- 3.2 Blatt 01: Reguläre Sprachen . . . 64
  - 3.2.1 Sprachen von regulären Ausdrücken (1P) . . . 64
  - 3.2.2 Bezeichner in Programmiersprachen (3P) . . . 64
  - 3.2.3 Gleitkommazahlen in Programmiersprachen (2P) . . . 65
  - 3.2.4 Mailadressen? (1P) . . . 65
  - 3.2.5 Der zweitletzte Buchstabe (1P) . . . 65
  - 3.2.6 Sprache einer regulären Grammatik (2P) . . . 66
- 3.3 Blatt 02: CFG . . . 66
  - 3.3.1 PDA (2P) . . . 66
  - 3.3.2 Akzeptierte Sprache (1P) . . . 66
  - 3.3.3 Kontextfreie Sprache (1P) . . . 67
  - 3.3.4 Kontextfreie Grammatik (2P) . . . 67
  - 3.3.5 Kontextfreie Grammatik (4P) . . . 67

3.4 Blatt 03: ANTLR 68
3.4.1 Zusammenfassung 68
3.4.2 Methodik 68
3.4.3 Sprachdefinition 68
3.4.4 Aufgaben 71

1 Syllabus



1.1 Kursbeschreibung

Der Compiler ist das wichtigste Werkzeug in der Informatik. In der Königsdisziplin der Informatik schließt sich der Kreis, hier kommen die unterschiedlichen Algorithmen und Datenstrukturen und Programmiersprachenkonzepte zur Anwendung.

In diesem Modul geht es um ein grundlegendes Verständnis für die wichtigsten Konzepte im Compilerbau. Wir schauen uns dazu relevante aktuelle Tools und Frameworks an und setzen diese bei der Erstellung eines kleinen Compiler-Frontends für C++ ein.

1.2 Überblick Modulinhalte

Die folgenden Punkte werden wir im Laufe des Semesters diskutieren:

1. Lexikalische Analyse: Scanner/Lexer
   - Reguläre Sprachen
   - Generierung mit ANTLR
2. Syntaxanalyse: Parser
   - Kontextfreie Grammatiken (CFG)
   - LL-Parser (Top-Down-Parser)
   - Generierung mit ANTLR
3. Semantische Analyse: Attributierte Grammatiken und Symboltabellen

   - Namen und Scopes
   - Typen, Klassen, Polymorphie
4. Zwischencode: Intermediate Representation (IR), LLVM-IR
5. Backend:
   - LLVM
   - Interpreter: AST-Traversierung
6. C++ als zu verarbeitende Programmiersprache

1.3 Team

- BC George
- Carsten Gips (Sprechstunde nach Vereinbarung)

1.4 Kursformat

Vorlesung (2 SWS)	Praktikum (2 SWS)
Mi, 08:00 - 09:30 Uhr (online)	S3, G1: Di, 11:30 - 13:00 Uhr (online/Präsenz J101)
(Carsten: *Flipped Classroom*, BC: *Vorlesung*)	S3, G2: Fr, 14:00 - 15:30 Uhr (online/Präsenz J101)
	S3, G3: Mo, 09:15 - 10:45 Uhr (online/Präsenz J101)
	(Carsten: *online*, BC: *Präsenz*)

Online-Sitzungen per Zoom (**Zugangsdaten siehe ILIAS IFM 3.1 CB (PO23, 3. Semester)**). Sie *können* hierzu den Raum J101 (siehe Stundenplan) nutzen.

1.5 Fahrplan

Hier finden Sie einen abonnierbaren Google Kalender IFM 3.1 CB (PO23, 3. Semester) mit allen Terminen der Veranstaltung zum Einbinden in Ihre Kalender-App.

Abgabe der Übungsblätter jeweils **Montag bis 09:00 Uhr** im ILIAS. Vorstellung der Lösung im jeweiligen Praktikum in der Abgabewoche.

Monat	Woche	Vorlesung	Lead	Abgabe Aufgabenblatt	Vorstellung Praktikum
Oktober	43	23.: CFG || B04 B04 B04	Carsten, BC		

Monat	Woche	Vorlesung	Lead	Abgabe Aufgabenblatt	Vorstellung Praktikum
	44	29.: **18:00 - 19:30 Uhr (online): Edmonton I: ANTLR + Live-Coding**	*Edmonton*		
	44	30.: Parser mit ANTLR	Carsten		
November	45	06.: *Dienstbesprechung*		04.: B01 CFG	~~04.~~ **07. 08:00** / 05. / 08. (BC, *Präsenz*)
	52	25.: *Weihnachtspause*			
Januar	01	01.: *Weihnachtspause*			
	02	08.: *Sprechstunde/Freies Arbeiten*	Carsten	06.: B02 C++	06. / 07. / 10. (Carsten, *online*)
	03	15.: *Freies Arbeiten/Puffer*			
	04	22.: **Parcoursprüfung: Station 2 B04 (VL- und Praktika-Slots, siehe Ankündigung)**	Carsten, BC	20.: B03 Mini-Projekt	20. / 21. / 24. (**15:00-16:30**) (Carsten/BC, *online*)
(Prüfungsphase I)	05	30.: **Feedback-Gespräche (15:30 - 18:00 Uhr, online)**			
(Prüfungsphase II)		**Parcoursprüfung: Do, 27. Mar 2025, 08-18 Uhr, mdl. Prüfung (alle Themen)** (je Prüfung ca. 45', Vergabe ca. 2 Wochen vorher)			
		TEST			

1.6 Prüfungsform, Note und Credits

Parcoursprüfung plus Testat, 5 ECTS (PO23)

- **Testat**: Vergabe der Credit-Points
  1. Mindestens 4 der Übungsblätter B01, B02, B03, B04 erfolgreich bearbeitet, **und**
  2. **aktive** Teilnahme an allen 3 Edmonton-Terminen.

  ("erfolgreich bearbeitet": Bearbeitung in 3er Teams, je mindestens 60% bearbeitet, fristgerechte Abgabe der Lösungen im ILIAS, Vorstellung der Lösungen im Praktikum)

1.6.1 Prüfung im ersten Zeitraum"

- **Stationen**:
  1. ILIAS-Test (einzeln, 20.11.)

2. Vorstellung Mini-Projekt B04 (3er Teams, letzte VL-Woche)

Note für das Modul: Beide Stationen ergeben zu **je 50%** *oder* in der Gewichtung **30 Punkte (Station I) und 50 Punkte (Station II)** die Gesamtnote (individuelle Günstigerprüfung).

Für Station I werden 3 Punkte Überhang gewährt: Von den 33 maximal erreichbaren Punkten werden 30 Punkte als 100% gewertet, darüber hinausgehende Punkte bleiben als Bonuspunkte erhalten.

1.6.2 Prüfung im zweiten Zeitraum”

- **Stationen**:

  1. Mündliche Prüfung (individuell, ca. 45 Minuten, zweiter Prüfungszeitraum)

  Die Note der mündlichen Prüfung ergibt die Gesamtnote.

1.7 Materialien

1. “**Compilers: Principles, Techniques, and Tools**”. Aho, A. V. und Lam, M. S. und Sethi, R. und Ullman, J. D. and Bansal, S., Pearson India, 2023. ISBN 978-9-3570-5488-1. Online über die O’Reilly-Lernplattform.
2. “**Crafting Interpreters**”. Nystrom, R., Genever Benning, 2021. ISBN 978-0-9905829-3-9. Online.
3. “**The Definitive ANTLR 4 Reference**”. Parr, T., Pragmatic Bookshelf, 2014. ISBN 978-1-9343-5699-9. Online über die O’Reilly-Lernplattform.
4. “Writing a C Compiler”. Sandler, N., No Starch Press, 2024. ISBN 978-1-0981-8222-9. Online über die O’Reilly-Lernplattform.

1.8 Förderungen und Kooperationen

1.8.1 Kooperation mit University of Alberta, Edmonton (Kanada)

Über das Projekt “We CAN virtuOWL” der Fachhochschule Bielefeld ist im Frühjahr 2021 eine Kooperation mit der University of Alberta (Edmonton/Alberta, Kanada) im Modul “Compilerbau” gestartet.

Wir freuen uns, auch in diesem Semester wieder drei gemeinsame Sitzungen für beide Hochschulen anbieten zu können. (Diese Termine werden in englischer Sprache durchgeführt.)

1.9 LICENSE

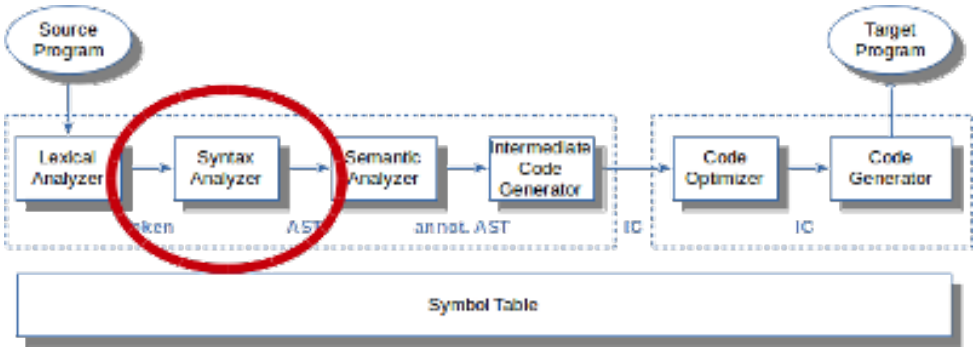
Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Hier steht vielleicht noch mehr - zentrales Readme mit detaillierter Lizenz und Credits. (Nicht mit generiertem Lizenz-Footer aus cb.yaml verwechseln.)

2 Vorlesungsunterlagen

2.1 Syntaktische Analyse

In der syntaktischen Analyse arbeitet ein Parser mit dem Tokenstrom, der vom Lexer kommt. Mit Hilfe einer Grammatik wird geprüft, ob gültige Sätze im Sinne der Sprache/Grammatik gebildet wurden. Der Parser erzeugt dabei den Parse-Tree. Man kann verschiedene Parser unterscheiden, beispielsweise die LL- und die LR-Parser.



2.1.1 CFG

Weitere Unterlagen

- Annotierte Folien: CFG, LL-Parser

2.1.1.1 Wiederholung

2.1.1.1.1 Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Wie sind DFAs und NFAs definiert?
- Was sind reguläre Ausdrücke?
- Was sind formale und reguläre Grammatiken?
- In welchem Zusammenhang stehen all diese Begriffe?
- Wie werden DFAs und reguläre Ausdrücke im Compilerbau eingesetzt?

2.1.1.2 Motivation

2.1.1.2.1 Wofür reichen reguläre Sprachen nicht? Für z. B. alle Sprachen, in deren Wörtern Zeichen über eine Konstante hinaus gezählt werden müssen. Diese Sprachen lassen sich oft mit Variablen im Exponenten beschreiben, die unendlich viele Werte annehmen können.

- $a^i b^{2*i}$ ist nicht regulär
- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär

- Wo finden sich die oben genannten Variablen bei einem DFA wieder?
- Warum ist die erste Sprache oben nicht regulär, die zweite aber?

2.1.1.2.2 Themen für heute

- PDAs: mächtiger als DFAs, NFAs
- kontextfreie Grammatiken und Sprachen: mächtiger als reguläre Grammatiken und Sprachen
- DPDAs und deterministisch kontextfreie Grammatiken: die Grundlage der Syntaxanalyse im Compilerbau
- Der Einsatz kontextfreier Grammatiken zur Syntaxanalyse mittels Top-Down-Techniken

2.1.1.2.3 Einordnung: Erweiterung der Automatenklasse DFA, um komplexere Sprachen als die regulären akzeptieren zu können Wir spendieren den DFAs einen möglichst einfachen, aber beliebig großen, Speicher, um zählen und matchen zu können. Wir suchen dabei konzeptionell die "kleinstmögliche" Erweiterung, die die akzeptierte Sprachklasse gegenüber DFAs vergrößert.

- Der konzeptionell einfachste Speicher ist ein Stack. Wir haben keinen wahlfreien Zugriff auf die gespeicherten Werte.
- Es soll eine deterministische und eine indeterministische Variante der neuen Automatenklasse geben.
- In diesem Zusammenhang wird der Stack auch Keller genannt.

2.1.1.2.4 Kellerautomaten (Push-Down-Automata, PDAs) **Def.:** Ein Kellerautomat (PDA) $P = (Q,\ \Sigma,\ \Gamma,\ \delta,\ q_0,\ \perp,\ F)$ ist ein Septupel mit:

Q:	eine endliche Menge von Zuständen
Σ:	eine endliche Menge von Eingabesymbolen
Γ:	ein endliches Kelleralphabet
δ:	die Übergangsfunktion $\delta : \mathrm{Q} \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$
q_0:	der Startzustand
$\perp \in \Gamma$:	der anfängliche Kellerinhalt, symbolisiert den leeren Keller ($\perp$ = bottom)
$F \subseteq Q$:	die Menge von Endzuständen

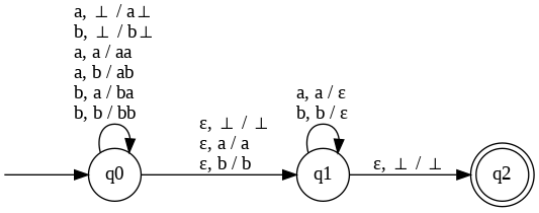
Abbildung 1: Definition eines PDAs

Ein PDA ist per Definition nichtdeterministisch und kann spontane Zustandsübergänge durchführen.

2.1.1.2.5 Was kann man damit akzeptieren? Strukturen mit paarweise zu matchenden Symbolen.

Bei jedem Zustandsübergang wird ein Zeichen (oder ϵ) aus der Eingabe gelesen, ein Symbol von Keller genommen. Diese und das Eingabezeichen bestimmen den Folgezustand und eine Zeichenfolge, die auf den Stack gepackt wird. Dabei wird ein Symbol, (z. B. eines, das später mit einem Eingabesymbol zu matchen ist,) auf den Stack gepackt. Soll das automatisch vom Stack genommene Symbol auf dem Stack bleiben, muss es wieder gepusht werden.

2.1.1.2.6 Beispiel Ein PDA für $L = \{ww^R \mid w \in \{a,b\}^*\}$:



2.1.1.2.7 Deterministische PDAs **Def.** Ein PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist *deterministisch* $:\Leftrightarrow$

- $\delta(q, a, X)$ hat höchstens ein Element für jedes $q \in Q, a \in \Sigma$ oder $(a = \epsilon$ und $X \in \Gamma)$.
- Wenn $\delta(q, a, x)$ nicht leer ist für ein $a \in \Sigma$, dann muss $\delta(q, \epsilon, x)$ leer sein.

Deterministische PDAs werden auch *DPDAs* genannt.

2.1.1.2.8 Der kleine Unterschied **Satz:** Die von DPDAs akzeptierten Sprachen sind eine echte Teilmenge der von PDAs akzeptierten Sprachen.

Die regulären Sprachen sind eine echte Teilmenge der von DPDAs akzeptierten Sprachen.

2.1.1.3 Kontextfreie Grammatiken und Sprachen

2.1.1.3.1 Kontextfreie Grammatiken **Def.** Eine *kontextfreie (cf-)* Grammatik ist ein 4-Tupel $G = (N, T, P, S)$ mit N, T, S wie in (formalen) Grammatiken und P ist eine endliche Menge von Produktionen der Form:

$X \to Y_1 \ldots Y_n$ mit $X \in N$, $Y_i \in (N \cup T)$.

2.1.1.3.2 Beispiel $S \to (S) \mid S^*.S \mid S * S$

Ableitungsbäume für $a + b * c$ sind definiert wie bei regulären Sprachen. Bei cf-Grammatiken nennt man die Ableitungsbäume oft *Parse trees*.

Hier entsteht ein Tafelbild.

2.1.1.3.3 Nicht jede kontextfreie Grammatik ist eindeutig **Def.:** Gibt es in einer von einer kontextfreien Grammatik erzeugten Sprache ein Wort, für das mehr als ein Ableitungsbaum existiert, so heißt diese Grammatik *mehrdeutig*. Anderenfalls heißt sie *eindeutig*.

Satz: Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.

Satz: Es gibt kontextfreie Sprachen, für die keine eindeutige Grammatik existiert.

2.1.1.3.4 Kontextfreie Grammatiken und PDAs **Satz:** Die kontextfreien Sprachen und die Sprachen, die von PDAs akzeptiert werden, sind dieselbe Sprachklasse.

Satz: Eine von einem DPDA akzeptierteSprache hat eine eindeutige Grammatik.

Vorgehensweise im Compilerbau: Eine Grammatik für die gewünschte Sprache definieren und schauen, ob sich daraus ein DPDA generieren lässt (automatisch).

2.1.1.4 Syntaxanalyse

2.1.1.4.1 Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (=Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

2.1.1.4.2 Syntax Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Diese vorgegebene Syntax wird im Compilerbau mit einer kontextfreien Grammatik beschrieben und mit einem sogenannten **Parser** analysiert.

Heute: LL-Parsing, mit dem man eine Teilmenge der eindeutigen kontextfreien Grammatiken syntaktich analysieren kann.

Dabei wird der Ableitungsbaum von oben nach unten aufgebaut.

2.1.1.4.3 Ziele der Syntaxanalyse

- Bestimmung der syntaktischen Struktur eines Programms
- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltablelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche.

2.1.1.5 LL(k)-Grammatiken

2.1.1.5.1 First-Mengen $S \to A \mid B \mid C$

Welche Produktion nehmen?

Wir brauchen die "terminalen k-Anfänge" von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren $First$ - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \backslash T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \overset{*}{\Rightarrow} w,$ mit $w \in T^*, First_k(w) = \{v\}\}$

2.1.1.5.2 Linksableitungen **Def.:** Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \overset{*}{\Rightarrow}_l \beta$.

2.1.1.5.3 LL(k)-Grammatiken **Def.:** Eine kontextfreie Grammatik $G = (N, T, P, S)$ ist genau dann eine *LL(k)*-Grammatik, wenn für alle Linksableitungen der Form:

$S \overset{*}{\Rightarrow}_l \ wA\gamma \Rightarrow_l w\alpha\gamma \overset{*}{\Rightarrow}_l wx$

und

$S \overset{*}{\Rightarrow}_l wA\gamma \Rightarrow_l w\beta\gamma \overset{*}{\Rightarrow}_l wy$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$\alpha = \beta$

2.1.1.5.4 LL(1)-Grammatiken Hier entsteht ein Tafelbild.

2.1.1.5.5 LL(k)-Sprachen Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von *LL(k+1)*-Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine *LL(1)* - Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen *LL(1)* - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine LL-Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

2.1.1.5.6 Was brauchen wir zur Erzeugung eines LL(k)-Parsers?

- eine *LL(k)*-Grammatik
- die $First_k$-Mengen der rechten Seiten aller Produktionsregeln
- die $Follow_k$-Mengen aller Nichtterminale und der rechten Seiten aller Produktionsregeln
- das Endezeichen $\perp$ hinter dem Eingabewort

Def.: Wir definieren $Follow$ - Mengen einer Grammatik wie folgt:

$Follow_k(\beta) = \{w \in T^* \mid \exists \alpha, \gamma \in (N \cup T)^*$ mit $S \overset{*}{\Rightarrow}_l \ \alpha\beta\gamma$ und $w \in First_k(\gamma)\}$

2.1.1.5.7 Beispiel: First- und Follow-Mengen Hier entsteht ein Tafelbild.

2.1.1.5.8 Algorithmus: Konstruktion einer LL-Parsertabelle **Eingabe:** Eine Grammatik $G = (N, T, P, S)$

Ausgabe: Eine Parsertabelle P

```
for each production X → α
   for each a ∈ First(α)
      add X → α to P[X, a]

   if ϵ ∈ First(α)
      for each b ∈ Follow(α)
         add X → α to P[X, b]
         if ϵ ∈ First(α) and ⊥ ∈ Follow(X)
            add X → α to P[X, ⊥]
```

Abbildung 2: Algorithmus zur Generierung einer LL-Parsertabelle

Statt $First_1(\alpha)$ wird oft nur $First(\alpha)$ geschrieben.

2.1.1.5.9 Beispiel: LL-Parsertabellen Hier entsteht ein Tafelbild.

2.1.1.5.10 LL-Parser Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt. Diesen Stack kann man auch "selbst programmieren", d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

2.1.1.5.11 Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA **Eingabe:** Eine Grammatik $G = (N, T, P, S)$, eine Parsertabelle P mit "$w \perp$" als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w, Fehler sonst

```
a = next_token()
X = top of stack  // entfernt X vom Stack

while X ≠⊥

   if X = a
      a = next_token()

   else if X ∈ T
      error

   else if P[X,a] leer
      error

   else if P[X,a] = X → Y1Y2...Yk
      process_production(X → Y1Y2...Yk)
      push(Y1Y2...Yk) //Y1 = top of stack

   X = top of stack
```

Abbildung 3: Algorithmus zum tabellengesteuerten LL-Parsen

2.1.1.5.12 Beispiel: LL-Parsen Hier entsteht ein Tafelbild.

2.1.1.5.13 Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Format für die Weiterverarbeitung:
  - Ableitungsbaum oder Syntaxbaum oder Parse Tree
  - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, . . .)

2.1.1.6 Wrap-Up

2.1.1.6.1 Das sollen Sie mitnehmen

- Die Struktur von gängigen Programmiersprachen lässt sich nicht mit regulären Ausdrücken beschreiben und damit nicht mit DFAs akzeptieren.
- Das Automatenmodell der DFAs wird um einen endlosen Stack erweitert, das ergibt PDAs.
- Kontextfreie Grammatiken (CFGs) erweitern die regulären Grammatiken.
- Deterministisch parsebare Sprachen haben eine eindeutige kontextfreie Grammatik.
- Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.
- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: PDAs
- k1: Deterministische PDAs
- k1: Kontextfreie Grammatiken
- k1: Deterministisch kontextfreie Grammatiken
- k1: Top-Down-Analyse
- k1: LL-Parser
- k2: Zusammenhang zwischen PDAs und kontextfreien Grammatiken

2.1.2 Parser mit ANTLR generieren

TL;DR
Mit ANTLR kann aus einer Grammatik ein LL(*)-Parser generiert werden. Die Parser-Regeln in der Grammatik fangen dabei mit einem **Kleinbuchstaben** an (Erinnerung: Lexer-Regel starten mit einem Großbuchstaben).
Regeln haben einen Namen (linke Seite) und eine Produktion (rechte Seite). Dabei können beliebige Abfolgen von Lexer- und Parser-Regeln auf der rechten Seite einer Parser-Regel auftauchen. Die Token müssen jeweils matchen, die Parser-Regeln werden in einen Aufruf der jeweiligen generierten Funktion übersetzt.
Parser-Regeln können aus mehreren Alternativen bestehen, diese werden per `|` separiert. Dabei hat bei Mehrdeutigkeiten die erste passende Alternative Vorrang. Wie bei Lexer-Regeln können Teile per `?` ein- oder keinmal vorkommen, per `*` beliebig oft oder per `+` ein- oder mehrfach.
ANTLR erlaubt im Gegensatz zu allgemeinen LL-Parsern direkte Links-Rekursion. (Indirekte Links-Rekursion funktioniert allerdings nicht.)
Der von ANTLR generierte Parser erzeugt auf der Eingabe einen Parse-Tree, der die Strukturen der Grammatik widerspiegelt: Die Token bilden die Blätter und jede erfolgreich durchlaufene Parser-Regel bildet einen entsprechenden Knoten im Baum.
Für die Traversierung des Parse-Tree kann man die generierten Listener- oder Visitor-Klassen nutzen. Beim Einsatz der Listener nutzt man die vorgegebene Klasse `ParseTreeWalker`, die mit dem Parse-Tree und dem Listener den Baum per Tiefensuche traversiert und immer die jeweiligen `enterRegel`- und `exitRegel`-Methoden aufruft. Beim Visitor muss die Traversierung selbst erledigt werden, hier steht die aus der Klassenhierarchie geerbte Methode `visit` als Startpunkt zur Verfügung. In dieser Methode wird basierend auf dem Knotentyp die in den Visitor-Klassen implementierte `visitRegel`-Methode aufgerufen und man muss darauf achten, die Kindknoten durch passende Aufrufe zu traversieren. Sowohl bei den generierten Listener- als auch den Visitor-Klassen kann man die leeren Defaultmethoden bei Bedarf selbst überschreiben. Für den Zugriff auf die Regel-Elemente werden die sogenannten Kontextobjekte als Parameter übergeben.
Benannte Alternativen und Regel-Elemente sind nützlich, weil für die benannten Alternativen zusätzliche Kontextklassen erzeugt werden, über die dann auf die Bestandteile der Alternativen zugegriffen werden kann. Außerdem werden zusätzlich passende `enterAlternative`- und `exitAlternative`- bzw. `visitAlternative`-Methoden generiert. Für benannte Regel-Elemente wird ein entsprechend benanntes Attribut im Kontextobjekt angelegt, welches `public` sichtbar ist.

Videos

- VL Parser mit ANTLR (YT)
- Demo ANTLR Parser (YT)
- VL Parser mit ANTLR (HSBI)

2.1.2.1 Hello World

```
grammar Hello;

start : stmt* ;

stmt  : ID '=' expr ';' | expr ';' ;

expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;

atom  : ID | NUM ;

ID    : [a-z][a-zA-Z]* ;
NUM   : [0-9]+ ;
WS    : [ \t\n]+ -> skip ;
```

Konsole: Hello (grun, Parse-Tree)

2.1.2.1.1 Starten des Parsers

1. Grammatik übersetzen und Code generieren: `antlr Hello.g4`
2. Java-Code kompilieren: `javac *.java`
3. Parser ausführen:
   - `grun Hello start -tree` oder `grun Hello start -gui` (Grammatik "Hello", Startregel "start")
   - Alternativ mit kleinem Java-Programm:

```java
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
import org.antlr.v4.runtime.tree.ParseTree;

public class Main {
    public static void main(String[] args) throws Exception {
        HelloLexer lexer = new HelloLexer(CharStreams.fromStream(System
            .in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        HelloParser parser = new HelloParser(tokens);

        ParseTree tree = parser.start();  // Start-Regel
        System.out.println(tree.toStringTree(parser));
    }
}
```

2.1.2.1.2 Startregeln

- `start` ist eine Parser-Regel => Eine Parser-Regel pro Grammatik wird benötigt, damit man den generierten Parser am Ende auch starten kann …
- Alle Regeln mit kleinem Anfangsbuchstaben sind Parser-Regeln
- Alle Regeln mit großem Anfangsbuchstaben sind Lexer-Regeln

2.1.2.1.3 Formen der Subregeln

```
stmt  : ID '=' expr ';' ;
```

Um die Regel `stmt` anwenden zu können, müssen alle Elemente auf der rechten Seite der Regel erfüllt werden. Dabei müssen die Token wie `ID`, = und `;` matchen und die Subregel `expr` muss erfüllt werden können. Beachten Sie das abschließende Semikolon am Ende einer ANTLR-Regel!

```
stmt  : ID '=' expr ';' | expr ';' ;
```

Alternativen werden durch ein | getrennt. Hier muss genau eine Alternative erfüllt werden. Falls nötig, trennt man die Alternativen durch Einschließung in runden Klammern vom Rest der Regel ab: `r : a (b | c)d ;`.

```
expr  : term ('+' term)* ;
```

Der durch den * gekennzeichnete Teil kann beliebig oft vorkommen oder auch fehlen. Bei einem + müsste der Teil mind. einmal vorkommen und bei einem ? entsprechend einmal oder keinmal.

Auch hier kann man die Operatoren durch ein zusätzliches ? auf non-greedy umschalten (analog zu den Lexer-Regeln).

(vgl. github.com/antlr/antlr4/blob/master/doc/parser-rules.md)

2.1.2.1.4 Reihenfolge in Grammatik definiert Priorität

Falls mehr als eine Parser-Regel die selbe Input-Sequenz matcht, löst ANTLR diese Mehrdeutigkeit auf, indem es die erste Alternative nimmt, die an der Entscheidung beteiligt ist.

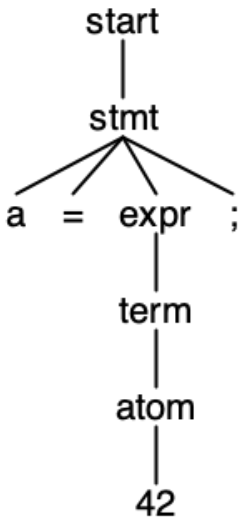
```
start : stmt ;

stmt  : expr | ID  ;
expr  : ID   | NUM ;
```

Bei der Eingabe “foo” würde die Alternative `ID` in der Regel `expr` “gewinnen”, weil sie in der Grammatik vor der Alternative `ID` in der Regel `stmt` kommt und damit Vorrang hat.

2.1.2.1.5 Parse-Tree

Betrachten wir erneut die obige Grammatik.

Die Eingabe von “`a = 42;`” führt zu folgendem Parse-Tree:



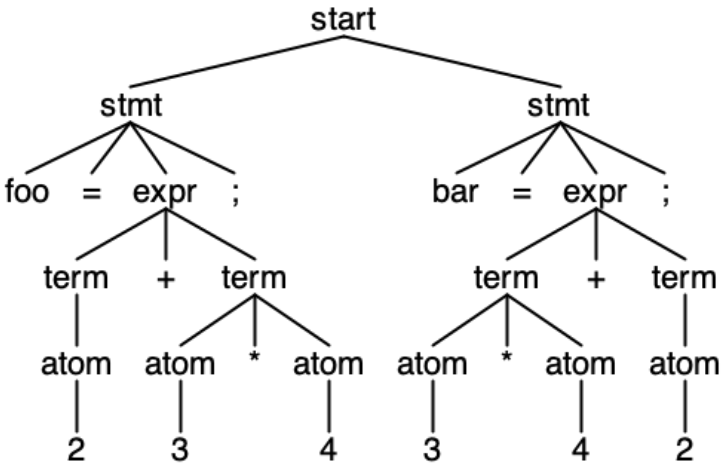
Diese Eingabe führt zur Erkennung der Token `[ID, WS, =, WS, NUM, ;]`, wobei die `WS`-Token verworfen werden und der Parser den Tokenstream `[ID, =, NUM, ;]` erhält.

Die Startregel hat auf der rechten Seite kein oder mehrere `stmt`-Regeln. Die `stmt`-Regel fordert auf der rechten Seite entweder die Token `ID`und = sowie die Regel `expr` gefolgt vom Token `;`, oder die Regel `expr` gefolgt vom Token `;`. In unserem Beispiel kann für das “a” das Token `ID` produziert werden, das “=” matcht ebenfalls. Die “42” wird erklärt, indem für `expr` ein `term` und dort ein `atom` aufgerufen wird. Für das `atom` muss entweder ein Token `ID` oder `NUM` als nächstes Token kommen - hier wird die “42” wird als Token `NUM` verarbeitet. Da die weiteren Regelteile in `term` und `expr` optional sind, haben wir damit ein `expr` erfüllt und das nachfolgende `;`-Token schließt die erste Alternative der Regel `stmt` erfolgreich ab.

Im entstehenden Parse-Tree sind diese Abläufe und grammatikalischen Strukturen direkt erkennbar. Jede erfolgreich durchlaufene Parserregel wird zu einem Knoten im Parse-Tree. Die Token werden als Terminale (Blätter) in den Baum eingehängt.

Anmerkung: Der Parse-Tree ist das Ergebnis der Parsers-Phase im Compiler und dient damit als Input für die folgenden Compilerstufen. In der Regel benötigt man die oft recht komplexen Strukturen aber später nicht mehr und vereinfacht den Baum zu einem *Abstract Syntax Tree* (AST). Im Beispiel könnte man den Zweig `stmt - expr - term - atom - 42` zu `stmt - 42` vereinfachen.

Betrachten wir nun die Eingabe `foo = 2+3*4; bar = 3*4+2;`. Diese führt zu folgendem Parse-Tree:



Wie man sehen kann, sind in der Grammatik die üblichen Vorrangregeln für die Operationen + und * berücksichtigt - die Multiplikation wird in beiden Fällen korrekt "unter" der Addition im Baum eingehängt.

2.1.2.1.6 To EOF not to EOF? Startregeln müssen nicht unbedingt den gesamten Input "konsumieren". Sie müssen per Default nur eine der Alternativen in der Startregel erfüllen.

Betrachten wir noch einmal einen leicht modifizierten Ausschnitt aus der obigen Grammatik:

```
1  start : stmt ;
```

Die Startregel wurde so geändert, dass sie nur noch genau ein Statement akzeptieren soll.

In diesem Fall würde die Startregel bei der Eingabe "aa; bb;" nur den ersten Teil "aa;" konsumieren (als Token `ID`) und das folgende "bb;" ignorieren. Das wäre in diesem Fall aber auch kein Fehler.

Wenn der gesamte Eingabestrom durch die Startregel erklärt werden soll, dann muss das vordefinierte Token `EOF` am Ende der Startregel eingesetzt werden:

```
1  start : stmt EOF;
```

Hier würde die Eingabe "aa; bb;" zu einem Fehler führen, da nur der Teil "aa;" durch die Startregel abgedeckt ist (Token `ID`), und der Rest "bb;" zwar sogar ein gültiges Token wären (ebenfalls `ID` und `;`), aber eben nicht mehr von der Startregel akzeptiert. Durch das `EOF` soll die Startregel aber den gesamten Input konsumieren und erklären, was hier nicht geht und entsprechend zum Fehler führt.

(vgl. github.com/antlr/antlr4/blob/master/doc/parser-rules.md)

2.1.2.2 Expressions und Vorrang (Operatoren) Betrachten wir noch einmal den Ausschnitt für die Ausdrücke (*Expressions*) in der obigen Beispielgrammatik:

```
1  expr  : term ('+' term)* ;
2  term  : atom ('*' atom)* ;
```

```
3  atom  : ID ;
```

Diese typische, etwas komplex anmutende Struktur soll sicher stellen, dass die Vorrangregeln für Addition und Multiplikation korrekt beachtet werden, d.h. dass `2+3*4` als `2+(3*4)` geparst wird und nicht fälschlicherweise als `(2+3)*4` erkannt wird.

Zusätzlich muss bei LL-Parsern Links-Rekursion vermieden werden: Die Parser-Regeln werden in Funktionsaufrufe übersetzt, d.h. bei einer Links-Rekursion würde man die selbe Regel immer wieder aufrufen, ohne ein Token aus dem Token-Strom zu entnehmen.

ANTLR (ab Version 4) kann mit beiden Aspekten automatisch umgehen:

- ANTLR kann direkte Linksrekursion automatisch auflösen. Die Regel `r : r T U | V ;` kann also in ANTLR verarbeitet werden.
- ANTLR besitzt einen Mechanismus zur Auflösung von Mehrdeutigkeiten. Wie oben geschrieben, wird bei der Anwendbarkeit von mehreren Alternativen die erste Alternative genutzt.

Damit lässt sich die typische Struktur für Expression-Grammatiken deutlich lesbarer gestalten:

```
1  expr  : expr '*' expr
2        | expr '+' expr
3        | ID
4        ;
```

Die Regel `expr` ist links-rekursiv, was normalerweise bei LL-Parsern problematisch ist. ANTLR löst diese Links-Rekursion automatisch auf (vgl. github.com/antlr/antlr4/blob/master/doc/left-recursion.md).

Da bei Mehrdeutigkeit in der Grammatik, also bei der Anwendbarkeit mehrerer Alternativen stets die erste Alternative genommen wird, lassen sich die Vorrangregeln durch die Reihenfolge der Alternativen in der `expr`-Regel implementieren: Die Multiplikation hat Vorrang von der Addition, und diese hat wiederum Vorrang von einer einfachen `ID`.

2.1.2.2.1 Direkte vs. indirekte Links-Rekursion ANTLR kann nur *direkte* Links-Rekursion auflösen. Regeln wie `r : r T U | V ;` stellen in ANTLR also kein Problem dar.

Indirekte Links-Rekursion erkennt ANTLR dagegen nicht:

```
1  r : s T U | V ;
2  s : r W X ;
```

Hier würden sich die Regeln `r` und `s` gegenseitig aufrufen und kein Token aus dem Tokenstrom entfernen, so dass der generierte LL-Parser hier in einer Endlosschleife stecken bleiben würde. Mit indirekter Links-Rekursion kann ANTLR nicht umgehen.

2.1.2.2.2 Konflikte in Regeln Wenn mehrere Alternativen einer Regel anwendbar sind, entscheidet sich ANTLR für die erste Alternative.

Wenn sich mehrere Tokenregeln überlappen, “gewinnt” auch hier die zuerst definierte Regel.

```
def : 'func' ID '(' ')' block ;

FOR : 'for' ;
ID  : [a-z][a-zA-Z]* ;
```

Hier werden ein implizites Token `'func'` sowie die expliziten Token `FOR` und `ID` definiert. Dabei sind die Lexeme für `'func'` und `FOR` auch in `ID` enthalten. Dennoch werden `'func'` und `FOR` erkannt und nicht über `ID` gematcht, weil sie *vor* der Regel `ID` definiert sind.

Tatsächlich sortiert ANTLR die Regeln intern um, so dass alle Parser-Regeln *vor* den Lexer-Regeln definiert sind. Die impliziten Token werden dabei noch vor den expliziten Token-Regeln angeordnet. Im obigen Beispiel hat also `'func'` eine höhere Priorität als `FOR`, und `FOR` hat eine höhere Priorität als `ID`. Aus diesem Grund gibt es die Konvention, die Parser-Regeln in der Grammatik vor den Lexer-Regeln zu definieren - dies entspricht quasi der Anordnung, die ANTLR bei der Verarbeitung sowieso erzeugen würde.

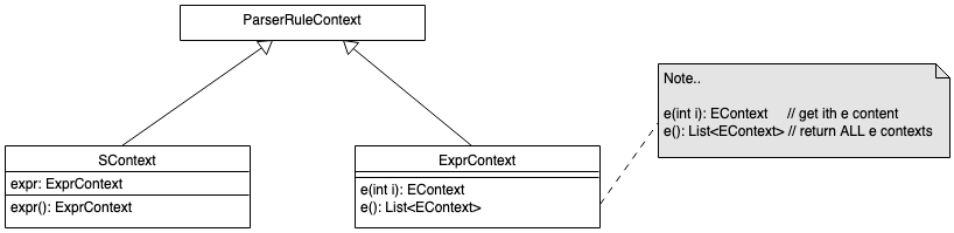
Aus diesem Grund würde auch eine Umsortierung der obigen Grammatik funktionieren:

```
FOR : 'for' ;
ID  : [a-z][a-zA-Z]* ;

def : 'func' ID '(' ')' block ;
```

Intern würde ANTLR die Parser-Regel `def` wieder vor den beiden Lexer-Regeln anordnen, und zwischen den Parser-Regeln und den Lexer-Regeln die impliziten Token (hier `'func'`).

2.1.2.3 Kontext-Objekte für Parser-Regeln

```
s    : expr         {List<EContext> x = $expr.ctx.e();}
     ;
expr : e '*' e ;
```



Jede Regel liefert ein passend zu dieser Regel generiertes Kontext-Objekt zurück. Darüber kann man das/die Kontextobjekt(e) der Sub-Regeln abfragen.

Die Regel `s` liefert entsprechend ein `SContext`-Objekt und die Regel `expr` liefert ein `ExprContext`-Objekt zurück.

In der Aktion fragt man das Kontextobjekt über `ctx` ab, in den Listener- und Visitor-Methoden erhält man die Kontextobjekte als Parameter.

Für einfache Regel-Aufrufe liefert die parameterlose Methode nur ein einziges Kontextobjekt (statt einer Liste) zurück.

Anmerkung: ANTLR generiert nur dann *Felder* für die Regel-Elemente im Kontextobjekt, wenn diese in irgendeiner Form referenziert werden. Dies kann beispielsweise durch Benennung (Definition eines Labels, siehe nächste Folie) oder durch Nutzung in einer Aktion (siehe obiges Beispiel) geschehen.

2.1.2.4 Benannte Regel-Elemente oder Alternativen

```
stat  : 'return' value=e ';'     # Return
      | 'break' ';'              # Break
      ;
```

```
public static class StatContext extends ParserRuleContext { ... }
public static class ReturnContext extends StatContext {
    public EContext value;
    public EContext e() { ... }
}
public static class BreakContext extends StatContext { ... }
```

Mit `value=e` wird der Aufruf der Regel `e` mit dem Label `value` belegt, d.h. man kann mit `$e.text` oder `$value.text` auf das `text`-Attribut von `e` zugreifen. Falls es in einer Produktion mehrere Aufrufe einer anderen Regel gibt, **muss** man für den Zugriff auf die Attribute eindeutige Label vergeben.

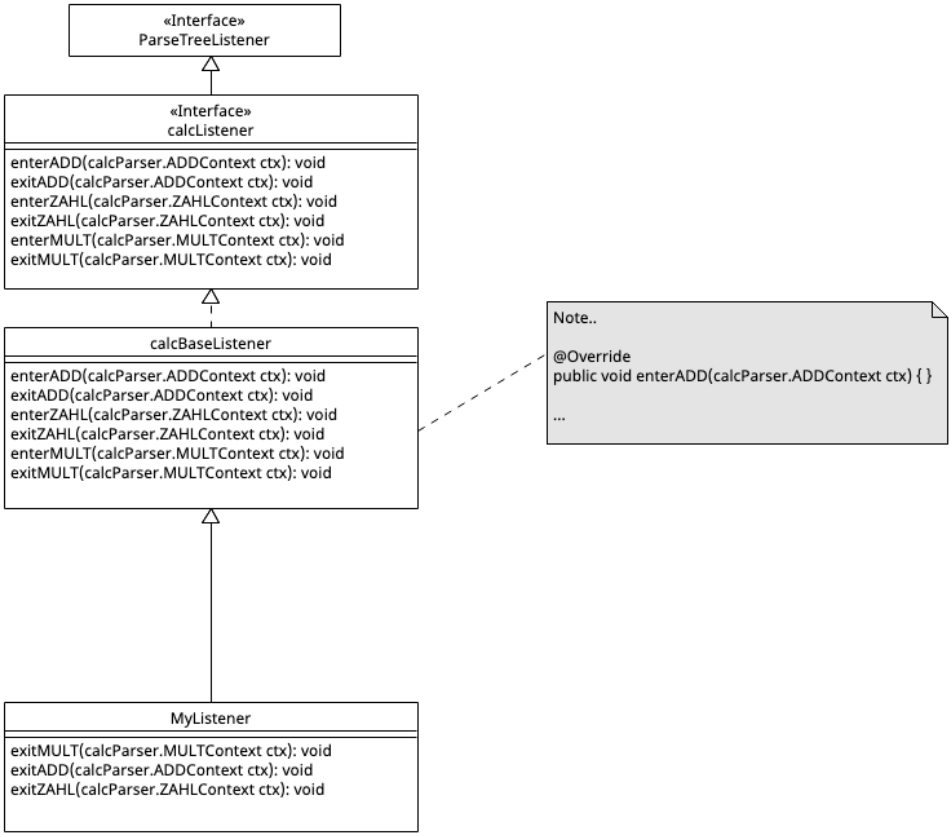
Analog wird für die beiden Alternativen je ein eigener Kontext erzeugt.

2.1.2.5 Arbeiten mit ANTLR-Listeners ANTLR (generiert auf Wunsch) zur Grammatik passende Listener (Interface und leere Basisimplementierung). Beim Traversieren mit dem Default-`ParseTreeWalker` wird der Parse-Tree mit Tiefensuche abgelaufen und jeweils beim Eintritt in bzw. beim Austritt aus einen/m Knoten der passende Listener mit dem passenden Kontext-Objekt aufgerufen.

Damit kann man die Grammatik "für sich" halten, d.h. unabhängig von einer konkreten Zielsprache und die Aktionen über die Listener (oder Visitors, s.u.) ausführen.

```
expr : e1=expr '*' e2=expr       # MULT
     | e1=expr '+' e2=expr       # ADD
     | DIGIT                     # ZAHL
     ;
```

ANTLR kann zu dieser Grammatik `calc.g4` einen passenden Listener (Interface `calcListener`) generieren (Option `-listener` beim Aufruf von `antlr`). Weiterhin generiert ANTLR eine leere Basisimplementierung (Klasse `calcBaseListener`):



(Nur “interessante” Methoden gezeigt.)

Von dieser Basisklasse leitet man einen eigenen Listener ab und implementiert die Methoden, die man benötigt.

```
public static class MyListener extends calcBaseListener {
    public void exitMULT(calcParser.MULTContext ctx) {
        ...
    }
    public void exitADD(calcParser.ADDContext ctx) {
        ...
    }
    public void exitZAHL(calcParser.ZAHLContext ctx) {
        ...
    }
}
```

Anschließend baut man das alles in eine Traversierung des Parse-Trees ein:

```
public class TestMyListener {
    public static class MyListener extends calcBaseListener {
        ...
    }

    public static void main(String[] args) throws Exception {
        calcLexer lexer = new calcLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
```

```
9          calcParser parser = new calcParser(tokens);
10
11         ParseTree tree = parser.s();    // Start-Regel
12
13         ParseTreeWalker walker = new ParseTreeWalker();
14         MyListener eval = new MyListener();
15         walker.walk(eval, tree);
16     }
17 }
```

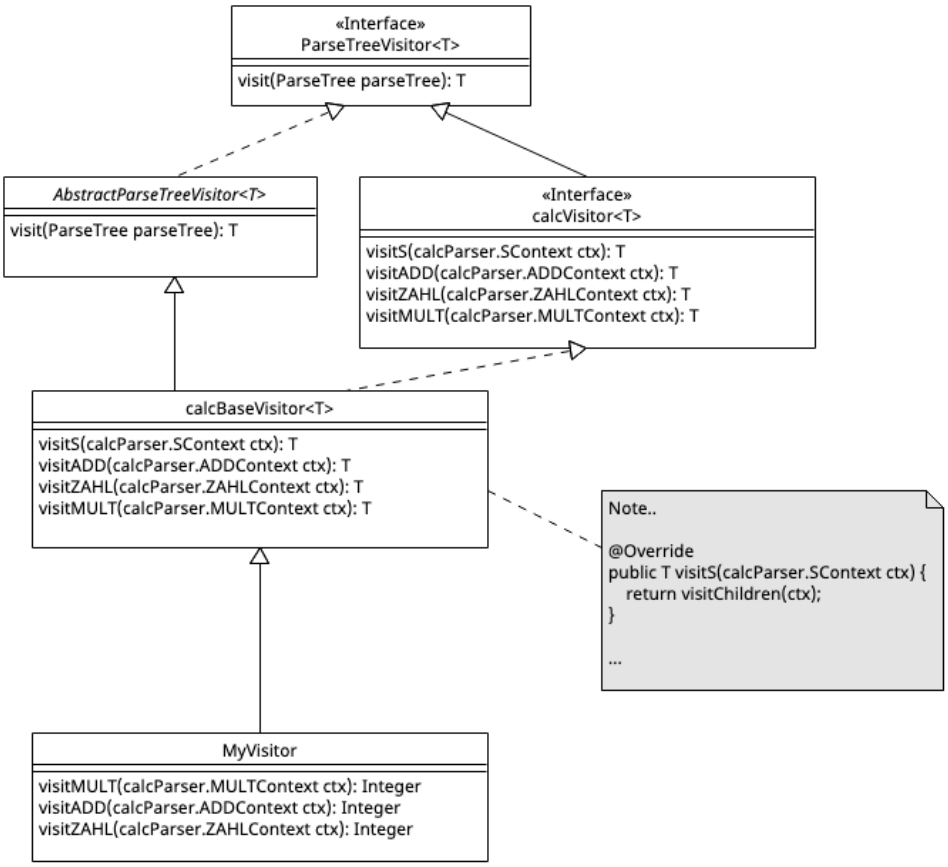
Beispiel: TestMyListener.java und calc.g4

2.1.2.6 Arbeiten mit dem Visitor-Pattern ANTLR (generiert ebenfalls auf Wunsch) zur Grammatik passende Visitoren (Interface und leere Basisimplementierung).

Hier muss man im Gegensatz zu den Listeners allerdings selbst für eine geeignete Traversierung des Parse-Trees sorgen. Dafür hat man mehr Freiheiten im Vergleich zum Einsatz von Listeners, insbesondere im Hinblick auf Rückgabewerte.

```
1 expr : e1=expr '*' e2=expr      # MULT
2      | e1=expr '+' e2=expr      # ADD
3      | DIGIT                    # ZAHL
4      ;
```

ANTLR kann zu dieser Grammatik einen passenden Visitor (Interface `calcVisitor<T>`) generieren (Option `-visitor` beim Aufruf von `antlr`). Weiterhin generiert ANTLR eine leere Basisimplementierung (Klasse `calcBaseVisitor<T>`):



(Nur “interessante” Methoden gezeigt.)

Von dieser Basisklasse leitet man einen eigenen Visitor ab und überschreibt die Methoden, die man benötigt. Wichtig ist, dass man selbst für das “Besuchen” der Kindknoten sorgen muss (rekursiver Aufruf der geerbten Methode `visit()`).

```
public static class MyVisitor extends calcBaseVisitor<Integer> {
    public Integer visitMULT(calcParser.MULTContext ctx) {
        return ...
    }
    public Integer visitADD(calcParser.ADDContext ctx) {
        return ...
    }
    public Integer visitZAHL(calcParser.ZAHLContext ctx) {
        return ...
    }
}
```

Anschließend baut man das alles in eine manuelle Traversierung des Parse-Trees ein:

```
public class TestMyVisitor {
    public static class MyVisitor extends calcBaseVisitor<Integer> {
        ...
    }

```

```
    public static void main(String[] args) throws Exception {
        calcLexer lexer = new calcLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        calcParser parser = new calcParser(tokens);

        ParseTree tree = parser.s();    // Start-Regel

        MyVisitor eval = new MyVisitor();
        eval.visit(tree);
    }
}
```

Beispiel: TestMyVisitor.java und calc.g4

2.1.2.7 Eingebettete Aktionen und Attribute

```
s    : expr                      {System.err.println($expr.v);}
     ;

expr returns [int v]
     : e1=expr '*' e2=expr       {$v = $e1.v * $e2.v;}
     ;
```

Auch die Parser-Regeln können mit eingebetteten Aktionen ergänzt werden, die in die (für die jeweilige Regel) generierte Methode eingefügt werden und bei erfolgreicher Anwendung der Parser-Regel ausgeführt werden.

Über `returns [int v]` fügt man der Regel `expr` ein Attribut `v` (Integer) hinzu, welches man im jeweiligen Kontext abfragen bzw. setzen kann (agiert als Rückgabewert der generierten Methode). Auf diesen Wert kann in den Aktionen mit `$v` zugegriffen werden.

Anmerkung: Durch den Einsatz von eingebetteten Aktionen und Attributen wird die Grammatik abhängig von der Zielsprache des generierten Lexers/Parsers!

2.1.2.8 Ausblick Damit haben wir die sprichwörtliche “Spitze des Eisbergs” gesehen. Mit ANTLR sind noch viele weitere Dinge möglich. Bitte nutzen Sie aktiv die Dokumentation auf github.com/antlr/antlr4.

2.1.2.9 Wrap-Up Parser mit ANTLR generieren: Parser-Regeln werden mit **Kleinbuchstaben** geschrieben

- Regeln können Lexer- und Parser-Regeln “aufrufen”
- Regeln können Alternativen haben
- Bei Mehrdeutigkeit: Vorrang für erste Alternative
- ANTLR erlaubt direkte Links-Rekursion
- ANTLR erzeugt Parse-Tree
- Benannte Alternativen und Regel-Elemente
- Traversierung des Parse-Tree: Listener oder Visitoren, Zugriff auf Kontextobjekte

Zum Nachlesen

- Parr (2014)

Lernziele

- k2: Aufbau der Parser-Regeln
- k3: Alternativen und optionale/mehrfache Regelteile in Parser-Regeln
- k3: Vorrang von Alternativen (bei Mehrdeutigkeiten)
- k3: Benannte Alternativen und Regel-Elemente
- k2: Aufbau des Parse-Tree
- k3: Umgang mit Kontext-Objekten
- k3: Traversierung des Parse-Tree mit den generierten Listenern oder Visitors

Challenges

Lexer und Parser mit ANTLR: Programmiersprache Lox

Betrachten Sie folgenden Code-Schnipsel in der Sprache "Lox":

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib(4);
```

Erstellen Sie für diese fiktive Sprache einen Lexer+Parser mit ANTLR. Implementieren Sie mit Hilfe des Parse-Trees und der Listener oder Visitoren einen einfachen Pretty-Printer.
(Die genauere Sprachdefinition finden Sie bei Bedarf unter craftinginterpreters.com/the-lox-language.html.)

2.2 Test Markdown

Hier werden alle relevanten Markdown-Features getestet

2.2.1 Test Markdown

TL;DR
Text für TL;DR …
In Parr (2014) wird geschrieben, blablablabla …
Wir können hier sowohl Inline-Math ($a^i b^{2*i}$) als auch Block-Math:

$$\Phi(\mathbf{g}_i) = F(\Gamma(\mathbf{g}_i)) - w \cdot \sum_j (Z_j(\Gamma(\mathbf{g}_i)))^2$$

Code sollte auch gehen: `inline`, aber auch block:

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib(4);
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. In eu vulputate nisl. Ut scelerisque magna eros, at semper lectus vulputate vitae. Nullam egestas tellus lorem, eget sodales mauris lacinia ut. Etiam a viverra ex. Nam eu nisl vel nisl cursus condimentum. Aliquam accumsan augue ut consequat viverra. Curabitur sagittis est mauris, at molestie arcu condimentum ut. Quisque efficitur porta maximus. Donec non leo est. Aenean interdum condimentum libero, ac cursus dolor condimentum in. Nullam lorem ex, iaculis a orci vitae, iaculis aliquam enim.
Quisque est lacus, pellentesque vitae fringilla vitae, bibendum sit amet dolor. Proin rutrum metus sit amet hendrerit lobortis. Fusce ut ultrices enim. Morbi a urna rutrum, fringilla augue mattis, mollis lacus. Pellentesque elementum vitae magna ac feugiat. Vestibulum et metus eget augue finibus fringilla ac at velit. Cras eleifend in nisl ac commodo. Orci varius natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Suspendisse id mi nec diam pellentesque varius. Morbi in consequat neque, at bibendum purus. Quisque a est bibendum, pellentesque quam ac, fermentum diam. Cras augue nibh, tincidunt eu mollis at, hendrerit nec ex. Integer condimentum neque velit, eget aliquam justo iaculis ac.
Nam consequat vehicula faucibus. In consequat sed lacus sed congue. Ut quis risus vel erat tincidunt molestie. Nullam nibh lorem, placerat et dignissim porttitor, mollis eget nisl. Mauris eu justo nisi. Sed viverra, enim in tincidunt blandit, dui eros mattis nulla, in elementum neque ex sed diam. Aliquam nec neque vitae sapien pulvinar tempus. Donec gravida interdum nunc sed feugiat. Curabitur finibus sed urna at rutrum. Maecenas mollis pulvinar lobortis. Mauris dignissim orci ut metus eleifend, eget porttitor purus rhoncus. Phasellus volutpat egestas odio mollis pharetra. Cras sem dolor, commodo eu lectus vel, ultrices fringilla tellus. Fusce eleifend orci sed porttitor imperdiet. Phasellus suscipit, est ut semper blandit, ligula dui scelerisque felis, varius tincidunt tellus est vitae lectus. Phasellus eu molestie urna.
Sed sed leo vestibulum, iaculis justo in, aliquet mauris. Nunc luctus, metus quis vulputate lobortis, libero lacus imperdiet turpis, sit amet hendrerit magna nisi at leo. Pellentesque vehicula, mauris nec varius tempus, odio diam hendrerit mi, id porttitor ex felis ut sem. Vivamus tellus lacus, vehicula a augue non, aliquet aliquam quam. Donec dapibus quam sed ante blandit, in imperdiet enim dapibus. Etiam imperdiet sapien nec quam lobortis aliquet. Nam est eros, luctus consequat nulla ac, dapibus efficitur augue. Morbi tempor ex sed consectetur pharetra. Morbi ullamcorper ac enim nec accumsan. Nulla vestibulum eu turpis et rutrum.
Donec laoreet lectus at laoreet condimentum. Nulla porttitor elit iaculis turpis vestibulum, eget lobortis dolor pretium. Nulla sollicitudin convallis ultrices. Integer porttitor a nunc a viverra. Vestibulum pulvinar a urna et posuere.
Duis sem nibh, consequat vitae ornare id, porttitor vel odio. Duis sit amet mi non odio accumsan condimentum sit amet non leo. Duis ut ligula ligula.
Phasellus sagittis non nisi eu ultricies. Donec quis ipsum at velit finibus mollis. Praesent facilisis blandit ligula. Pellentesque nec quam id neque lobortis egestas. Etiam nec risus feugiat, pulvinar mi in, vestibulum justo. Proin ac nisi scelerisque, bibendum tellus quis, lacinia leo. Vestibulum consequat fermentum est, eget cursus est efficitur ut. Fusce quis bibendum sapien. Vivamus vel nisl nulla. Nunc vehicula, odio ac ultricies iaculis, turpis turpis mollis dolor,

Videos

- VL Parser mit ANTLR (YouTube)
- Demo ANTLR Parser (YouTube)
- https://foo.bar.de
- VL Git Basics (HSBI Medienportal)

Weitere Unterlagen

- Folien (raw link)
- Folien

GH-Preview

2.2.1.1 Hello World

Hier ist normaler Markdown-Text, mit **fett** und auch *kursiv*.

- Stichpunkt 1
- Stichpunkt 2
- Stichpunkt 3

1. Aufzählung 1
2. Aufzählung 2
3. Aufzählung 3
    1. Unterpunkt 3.1
    2. Unterpunkt 3.2

Hier die Pandoc-Markdown mark-Erweiterung.

Hier nochmal eine Stichpunktliste, aber ohne Leerzeile davor (+`lists_without_preceding_blankline`):

- Stichpunkt 1
- Stichpunkt 2
- Stichpunkt 3

2.2.1.2 Math

2.2.1.2.1 Inline

$\mathbf{g} = (g_1, \ldots, g_m) \in \{0,1\}^m$

- $a^i b^{2*i}$ ist nicht regulär
- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär

2.2.1.2.2 Block

$$\Phi(\mathbf{g}_i) = F(\Gamma(\mathbf{g}_i)) - w \cdot \sum_j (Z_j(\Gamma(\mathbf{g}_i)))^2$$

$$p_{sel}(\mathbf{g}_k) = \frac{\Phi(\mathbf{g}_k)}{\sum_j \Phi(\mathbf{g}_j)}$$

$$g_i^{(t+1)} = \begin{cases} \neg g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ g_i^{(t)} & \text{sonst} \end{cases}$$

2.2.1.2.3 Known Problems

- VSCode Preview: `\mbox{ tanh }` => tanh => `\text{ tanh }` => tanh
- GH Preview:
  - `\phantom{xyz}` => => ?? => ??
  - `\operatorname{tanh}` => $\tanh$ => `\mathop{\text{tanh}}` => tanh

$$g_i^{(t+1)} = \begin{cases} \neg \quad g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ \quad g_i^{(t)} & \text{sonst} \end{cases}$$

Schwierig: In Pandoc-Markdown muss Mathe mit `$` oder `$$` eingeschlossen werden, unabhängig vom konkreten Inhalt. In LaTeX ist aber `\begin{eqnarray}` bereits der Beginn einer Mathe-Umgebung, d.h. hier wären extra `$$` falsch. Das muss per Filter korrigiert werden!

```
$$\begin{eqnarray}
S &\rightarrow& a A                          \nonumber \\
A &\rightarrow& d B \ | \ b A \ | \ c A  \nonumber \\
B &\rightarrow& a c \ | \ b C \ | \ c A  \nonumber \\
C &\rightarrow& \epsilon                     \nonumber
\end{eqnarray}$$
```

should become

$$\begin{eqnarray}
S &\rightarrow& aA \\
A &\rightarrow& dB \,|\, bA \,|\, cA \\
B &\rightarrow& ac \,|\, bC \,|\, cA \\
C &\rightarrow& \epsilon
\end{eqnarray}$$

2.2.1.2.4 Tests Inline Math

`array` as inline math:

$\begin{array}{ll} \neg \quad g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ \quad g_i^{(t)} & \text{sonst} \end{array}$

`eqnarray` as inline math:

$$\begin{array}{rcl} S & \rightarrow & aA \\ A & \rightarrow & dB \mid bA \mid cA \\ B & \rightarrow & ac \mid bC \mid cA \\ C & \rightarrow & \epsilon \end{array}$$

Block Math

`array` as block math:

$$\begin{array}{rll} \neg & g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ & g_i^{(t)} & \text{sonst} \end{array}$$

`eqnarray` as block math:

$$\begin{array}{rcl} S & \rightarrow & aA \\ A & \rightarrow & dB \mid bA \mid cA \\ B & \rightarrow & ac \mid bC \mid cA \\ C & \rightarrow & \epsilon \end{array}$$

Newline after $$

`array` as block math w/ newline:

$$\begin{array}{rll} \neg & g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ & g_i^{(t)} & \text{sonst} \end{array}$$

`eqnarray` as block math w/ newline:

$$\begin{array}{rcl} S & \rightarrow & aA \\ A & \rightarrow & dB \mid bA \mid cA \\ B & \rightarrow & ac \mid bC \mid cA \\ C & \rightarrow & \epsilon \end{array}$$

2.2.1.3 Links

2.2.1.3.1 Link to WWW craftinginterpreters.com/the-lox-language.html

2.2.1.3.2 Internal Links empty link

link target: .

selbe ebene: readme.md

unterordner: subfolder/foo.md

zurück nach oben I: ../02-parsing/antlr-parsing.md

zurück nach oben II: ../../homework/sheet01.md

2.2.1.4 Code

```
grammar Hello;

start : stmt* ;

stmt  : ID '=' expr ';' | expr ';' ;
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : ID | NUM ;

ID    : [a-z][a-zA-Z]* ;
NUM   : [0-9]+ ;
WS    : [ \t\n]+ -> skip ;
```

Java-Code kompilieren: `javac *.java`

Listing 1: The preprocessing step, cf. [@Diehl]

```
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
import org.antlr.v4.runtime.tree.ParseTree;

public class Main {
    public static void main(String[] args) throws Exception {
        HelloLexer lexer = new HelloLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        HelloParser parser = new HelloParser(tokens);

        ParseTree tree = parser.start();  // Start-Regel
        System.out.println(tree.toStringTree(parser));
    }
}
```

Code ohne alles

```
import org.antlr.v4.runtime.CharStreams;
import org.antlr.v4.runtime.CommonTokenStream;
import org.antlr.v4.runtime.tree.ParseTree;

public class Main {
    public static void main(String[] args) throws Exception {
        HelloLexer lexer = new HelloLexer(CharStreams.fromStream(System.in));
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        HelloParser parser = new HelloParser(tokens);

        ParseTree tree = parser.start();  // Start-Regel
        System.out.println(tree.toStringTree(parser));
    }
}
```

2.2.1.5 Images Figures (w/ caption) should be centered like in LaTeX. Inline images will appear as is (also like in LaTeX).

2.2.1.5.1 Tests GH vs. Docsify Div und Markdown-Link:

“B” (small)

(Bild geht auf GH/Docsify, `div`/`center` nur Docsify)

img (relativ):

(Bild und Skalierung geht auf GH, auf Docsify nur wenn direkt geladen)

img (https)

- `?raw=true`:
- `raw.githubusercontent.com`:

(Bild+Skalierung gehen auf GH und Docsify)

picture

- `?raw=true`:
- `raw.githubusercontent.com`:

(Bild+Skalierung+Darkmode gehen auf GH und Docsify)

picture mit `<p align>`

- `?raw=true`:

foo bar wuppie fluppie

- `raw.githubusercontent.com`:

foo bar wuppie fluppie

(Bild+Skalierung+Darkmode+Einrückung gehen auf GH und Docsify)

figures mit picture und p

- `?raw=true`:

lalelu … foo!

- `raw.githubusercontent.com`:

lalelu … foo!

(Bild+Skalierung+Darkmode gehen auf GH und Docsify; die Einrückung scheitert und bräuchte ein `<div>`, was aber auf GH nicht erlaubt ist.)

figures mit picture und div

- `?raw=true`:

lalelu … foo!

- `raw.githubusercontent.com`:

lalelu … foo!

(Bild+Skalierung+Darkmode gehen auf GH und Docsify; die Einrückung scheitert auf GH)

Really deep headings level (h4 to h7 - warning)

Markdown allows level 6 headings, max. When building the book version, we need to shift all headings, in this case by 3. So in this example all h3 headings will become h6 headings, h4 headings (like this one) would become h7 headings, which do not exist. Our filter should emit a warning here.

NB: The filter should also warn on h3 headings, as those will be demoted to (valid) h6 headings in the first step and due to `--shift-heading-level-by=1` in the next processing step would become h7 headings.

2.2.1.5.2 Images with Caption kleines Bild, keine Breiteangabe:

Abbildung 4: “B” (small)

Bild in Elternordner, keine Breiteangabe:

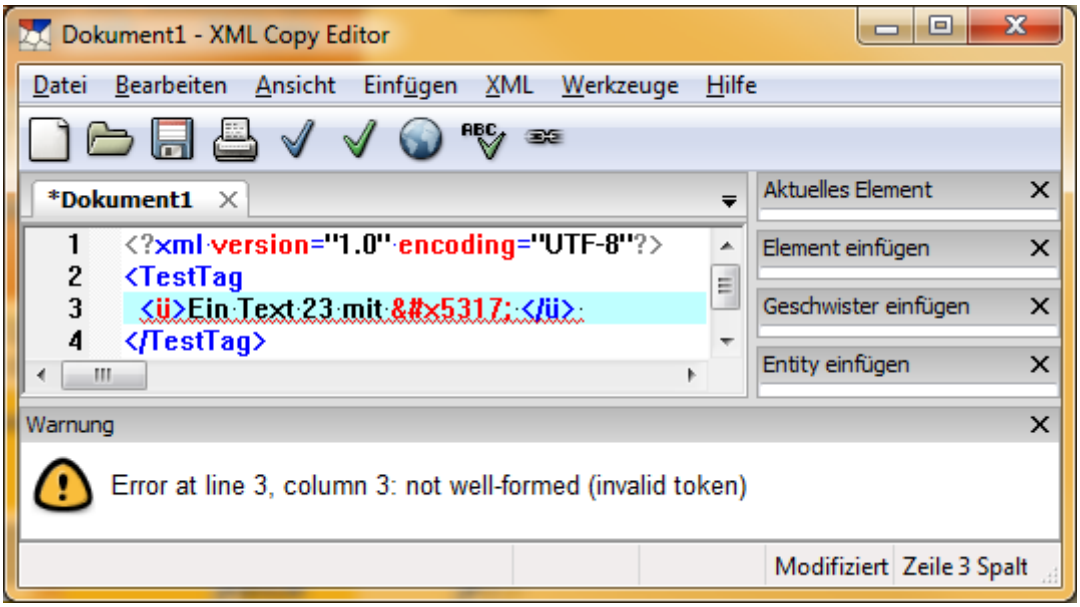


Abbildung 5: parsing error

Wahlpflichtmodul aus Liste 1 „Methoden der Informatik“ Compilerbau								**Kürzel CB**
Nr.	**Workload**	**Credit Points**	**Studien-semest**	**Häufigkeit**	**Sem.**	**Dauer**	**Art**	**Q-Niveau**
5.21	**150h**	**5**	5.Sem	jährlich	WiSe	1 Sem.	Wahl	B.Sc.
1	**Lehrveranstaltungs-art**		**Kontaktzeit**	**Selbst-studium**	**Lehrformen (Lernformen)**		**gepl. Gruppengr.**	**Sprache**
	Sem. Unterricht Praktikum / Seminar		2 SWS / 30h 2 SWS / 30h	45h 45h	Wird in LV bekannt gegeben.		60 15	deutsch und englisch

Abbildung 6: modulbeschreibung

---

kleines Bild, mit Titel und Breite:

Abbildung 7: “B”, width=“5%”

---

breites Bild, keine Breiteangabe:

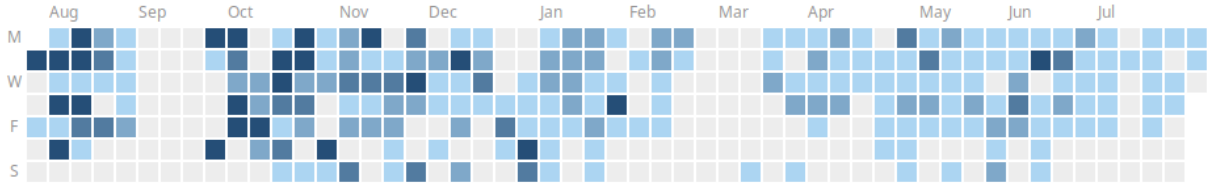


Abbildung 8: “wuppie” (wide)

---

breites Bild, mit Breiteangabe:

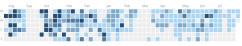


Abbildung 9: “wuppie”, width=“20%”

---

breites Bild über HTTP, keine Breiteangabe:

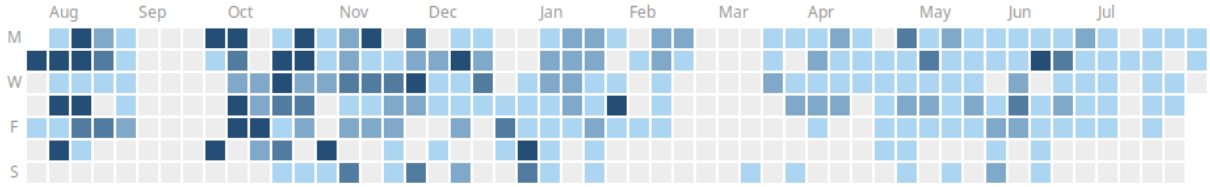


Abbildung 10: “wuppie” via web (raw)

---

breites Bild über HTTP mit `credits`-Span, keine Breiteangabe:

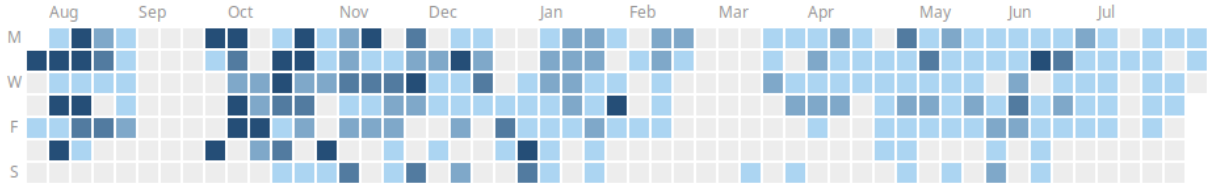


Abbildung 11: “wuppie” via web (raw) (Quelle: “FooFOOOO” by me on void.intern.com)

Quelle: “Foo” by me on void.extern.com

2.2.1.5.3 Images w/o Caption

ohne alles:

---

mit breitenangabe 5%:

---

mit breitenangabe (“width=15%”) und titel:

---

mit breitenangabe (“web_width=10%”) und titel:

---

mit breitenangabe (“width=5%” und “web_width=10%”) und titel:

2.2.1.5.4 Images w/ `img`-Code using “5%”:

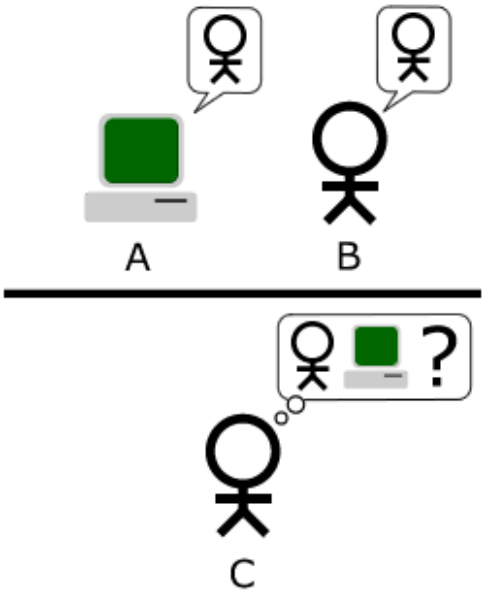
using “80px”:

mit div drumherum:

2.2.1.5.5 Known Problems

- In VSC preview as well as in LaTeX images via web like https://github.com/cagix/pandoc-thesis/blob/master/figs/wuppie.png do not work (**need to be "raw"**)

2.2.1.5.6 Images from Web (LaTeX backend) When converting to LaTeX (PDF, Beamer), Pandoc will attempt to download the referred images. However, recently a lot of sites deny this. It seems we need to set an user-agent.



Quelle: Turing Test version 3.png by Bilby on Wikimedia Commons (Public Domain)

The conversion above will fail with

```
Could not convert image /tmp/tex2pdf.-c72add2811a5b622/3
    d299b2a35aa76ba2acfef6d4887a64d6ea8e601.txt: Cannot load file
  Jpeg Invalid marker used
  PNG Invalid PNG file, signature broken
  Bitmap Invalid Bitmap magic identifier
  GIF Invalid Gif signature : Please
  HDR Invalid radiance file signature
  Tiff Invalid endian tag value
  TGA not enough bytes
Error producing PDF.
! LaTeX Error: Unknown graphics extension: .txt.

See the LaTeX manual or LaTeX Companion for explanation.
Type  H <return>  for immediate help.
 ...

l.1089 ...b2a35aa76ba2acfef6d4887a64d6ea8e601.txt}
```

unless there is a user-agent defined like

```
request-headers:
  - ["User-Agent", "Mozilla/5.0"]
```

Note: This is only needed, when Pandoc attempts to download the image locally, i.e. when converting to LaTeX based formats like PDF or Beamer. No need to set this for web based formats like GFM or Markdown since we just leave the link as it is.

(see https://github.com/cagix/pandoc-lecture-zen/issues/57) (see https://github.com/Artificial-Intelligence-HSBI-TDU/KI-Vorlesung/issues/455)

2.2.1.6 Tabellen

mit caption:

Tabelle 1: Tabelle als Markdown-Pipe-Table, vgl. (Abelson u. a. 1996)

Rechtsbündig	Linksbündig	Default	Zentriert
foo	foo	foo	foo
123	123	123	123
bar	bar	bar	bar

ohne caption:

Rechtsbündig	Linksbündig	Default	Zentriert
foo	foo	foo	foo
123	123	123	123
bar	bar	bar	bar

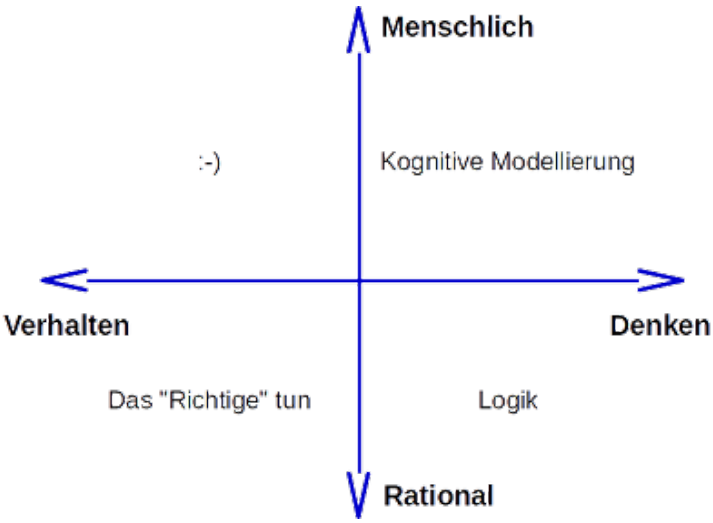
2.2.1.7 Zitieren, Quellen Normales Zitieren (Siek 2023) …

Mit Seitenangabe (Siek 2023, 111) oder Kapitel (Siek 2023, Kap. 111) …

Als Author-Zitat Siek (2023) …

2.2.1.8 GFM

2.2.1.8.1 Darkmode vs. Lightmode Diese Abbildung liegt in drei Varianten im Repo vor: transparenter Hintergrund, light, dark. Die Einbindung ist auf “transparent”. Scale ist auf 60%. Die Abbildung sollte sich automatisch umstellen.



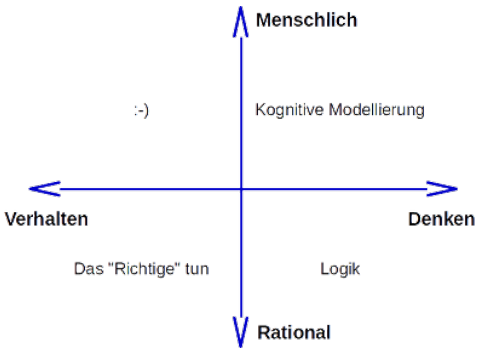
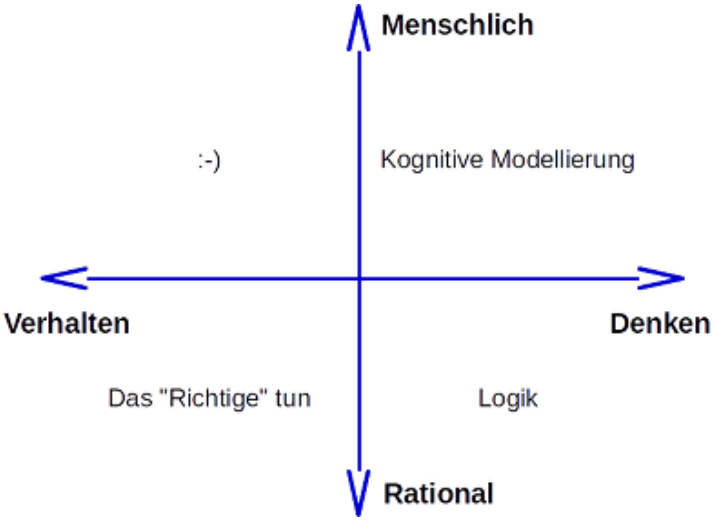
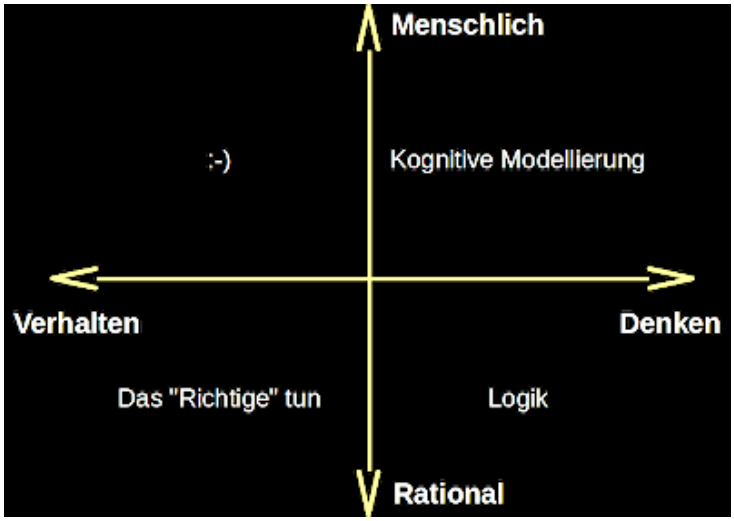


Abbildung 12: Hier könnte ihre Werbung stehen. (40%)

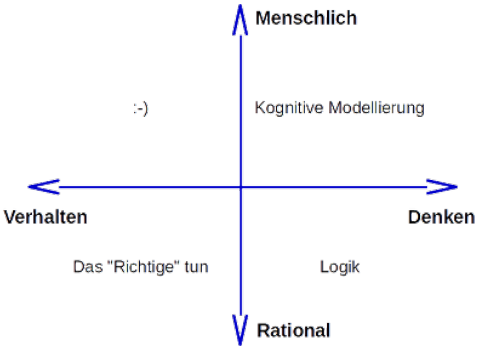
Nachfolgend die “light”-Variante direkt (60%):



Nachfolgend die “dark”-Variante direkt (60%):



Nachfolgend eine Abbildung, die es nicht in light/dark im Repo gibt. Wie verhält sich die Preview, so lange nicht beim Übersetzen die light- und dark-Varianten mit generiert werden? (40%)



Und noch einmal, diesmal mit Caption.

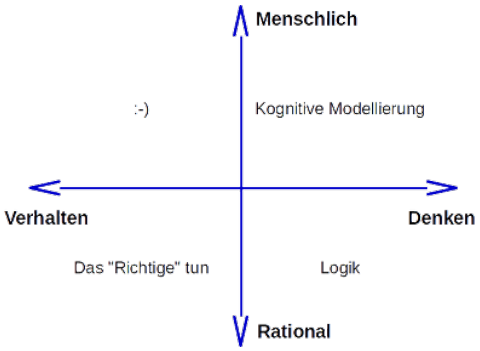


Abbildung 13: Hier könnte ihre Werbung stehen.

2.2.1.8.2 Details

Zusammenfassung: NIX :)

Lalelu …

2.2.1.8.3 Alert Extension

GH introduced “alerts” with distinctive styling, like

> Note
> NOTE: Foo bar, wuppie fluppie!

> Tip
> TIP: Foo bar, wuppie fluppie!

> Important
> IMPORTANT: Foo bar, wuppie fluppie!

> Warning
> WARNING: Foo bar, wuppie fluppie!

> Caution
> CAUTION: Foo bar, wuppie fluppie!

(see https://github.blog/changelog/2023-12-14-new-markdown-extension-alerts-provide-distinctive-styling-for-significant-content/)

(**formatted like in GH, needs –`f markdown+alerts` when reading from markdown to be recognized**)

---

gh alerts, written as nested pandoc divs (needs –`t markdown+alerts` when writing to markdown to produce the format above)

currently the markdown reader would not recognize the gh-alert properly if the extension `lists_without_preceding_blank` is activated (see https://github.com/jgm/pandoc/issues/11534), so this will fail until this issue is fixed:

> Note
> NOTE: Foo bar, wuppie fluppie!

> Tip
> TIP: Foo bar, wuppie fluppie!

> Important
> IMPORTANT: Foo bar, wuppie fluppie!

> Warning
> WARNING: Foo bar, wuppie fluppie!

> Caution
> CAUTION: Foo bar, wuppie fluppie!

Let's stick with Pandocs divs in Markdown content and use filters for export. This should work regardless the bugs relating to GH alterts in both the reader (https://github.com/jgm/pandoc/issues/11534) and the writer (https://github.com/jgm/pandoc/issues/11533), also when using "-f markdown+lists_without_preceding_blankline‘:

> NOTE: Foo bar, wuppie fluppie!

> TIP: Foo bar, wuppie fluppie!

> IMPORTANT: Foo bar, wuppie fluppie!

> WARNING: Foo bar, wuppie fluppie!

> CAUTION: Foo bar, wuppie fluppie!

(**formatted using custom divs in pandoc-markdown, needs custom lua-filter**)

(**usage in custom divs**)

This should work regardless the bugs relating to GH alterts in both the reader and the writer, also when using "-f markdown+lists_without_preceding_blankline‘:

TL;DR
foobar test

Challenges
prove that $E = mc^2$.

- Export to GH Markdown using "distinctive alerts"
- Export to Hugo using notice shortcode
- Export to Beamer using beamercolorbox (also beameruserguide.pdf; or `block`, `alertblock`, `examples` - cf. https://www.overleaf.com/learn/latex/Beamer%23Creating_a_table_of_contents)

This should probably be in line with #180 …

2.2.1.8.4 GH Shenanigans The GitHub Markdown parser has a really weird bug: as soon as there's inline math in a bullet point, any display math after that won't render properly in the whole bullet list.

Example

- Bullet point 1
- Bullet point 2 with block math (w/o blank line)

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- Bullet point 3 with block math and blank line

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Bullet point 4 with inline math (x_i) and block math and blank line

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Bullet point 5 with image

- Bullet point 6 with figure

Abbildung 14: image caption

- Bullet point 7 with code block

```
grammar Hello;

start : stmt* ;

stmt  : ID '=' expr ';' | expr ';' ;
expr  : term ('+' term)* ;
term  : atom ('*' atom)* ;
atom  : ID | NUM ;

ID    : [a-z][a-zA-Z]* ;
NUM   : [0-9]+ ;
```

```
WS     : [ \t\n]+ -> skip ;
```

foo bar wuppie fluppie - text below code block in bullet point 6

- simple bullet point

Tests

Using `bulletlist_displaymath.lua` to remove display math from bullet points.

This filter is intended as a quick-and-dirty workaround. If a display math element occurs in a bullet point, it is removed from the bullet point. The bullet list obtained so far is terminated, the display math element is inserted at the same AST level as the bullet list, and a new bullet list is started for the remaining bullet points.

For reasons that are not entirely clear, empty pandoc.Para elements are sometimes inserted into the bullet points. These are immediately removed again here in the filter.

Note: If a bullet point contains text followed by display math followed by text, the resulting order after the filter is no longer correct: First, the complete bullet point is emitted, followed by the display math.

```
-   wuppie $x_i$ fluppie:
    $$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$
-   foo bar

compare with

-   wuppie fluppie:
    $$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$
-   foo bar

compare with

-   wuppie $x_i$ fluppie:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

-   foo bar

compare with

-   wuppie $x_i$ fluppie:
    $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$
-   foo bar

compare with

*   wuppie $x_i$ fluppie $w_0$:
    *   bla bla bla

    *   blub $x$ blub blub

        $$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$
```

```
    *   brabbel brabbel brabbel

    *   blafasel $y$

        $$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

        blubfasel $z$

*   foobar

compare with

1.  wuppie $x_i$ fluppie $w_0$:

    $$h(\mathbf{x}^1) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

    *   bla bla bla

    *   blub $x$ blub blub

        $$h(\mathbf{x}^2) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

    *   brabbel brabbel brabbel

    *   blafasel $y$

        $$h(\mathbf{x}^3) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

        blubfasel $z$

2.  foobar
```

will be rendered as

- wuppie x_i fluppie:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- foo bar

compare with

- wuppie fluppie:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- foo bar

compare with

- wuppie x_i fluppie:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- foo bar

compare with

- wuppie x_i fluppie: $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$
- foo bar

compare with

- wuppie x_i fluppie w_0:
  - bla bla bla
  - blub x blub blub

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

  - brabbel brabbel brabbel
  - blafasel y

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

  blubfasel z
- foobar

compare with

1. wuppie x_i fluppie w_0:

$$h(\mathbf{x}^1) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

   - bla bla bla
   - blub x blub blub

$$h(\mathbf{x}^2) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

   - brabbel brabbel brabbel
   - blafasel y

$$h(\mathbf{x}^3) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

   blubfasel z
2. foobar

Test Case nn02-linear-regression.md

nn02-linear-regression.md

2.2.1.9 Filter for Slides and Handouts

Foo bar, wuppie fluppie! (NOTES)

2.2.1.10 Footnotes

Sometimes[1] we need some[2] footnotes.

2.2.1.10.1 Test for Docsify

~~Docsify seems to recognize footnotes even in inline code:~~

Zeichenkette	Beschreibt
^	Zeilenanfang
[abc]	“a” oder “b” oder “c”
[^abc]	alles außer “a”, “b” oder “c” (Negation)
^abc	test
[^abc]	test
[^abc]	test
[^ abc]	test
[a-zA-Z]	alle Zeichen von “a” bis “z” und “A” bis “Z” (Range)
[a-z&&[def]]	“d”,“e” oder “f” (Schnitt)
[a-z&&[^bc]]	“a” bis “z”, außer “b” und “c”: [ad-z] (Subtraktion)
[a-z&&[^m-p]]	“a” bis “z”, außer “m” bis “p”: [a-lq-z] (Subtraktion)

```
^                // Zeilenanfang
[abc]            // "a" oder "b" oder "c"
[^abc]           // alles außer "a", "b" oder "c" (Negation)
^abc             // test
[ ^abc]          // test
[ ^abc]          // test
[^ abc]          // test
[a-zA-Z]         // alle Zeichen von "a" bis "z" und "A" bis "Z" (Range)
[a-z&&[def]]     // "d","e" oder "f" (Schnitt)
[a-z&&[^bc]]     // "a" bis "z", außer "b" und "c": `[ad-z]` (Subtraktion)
[a-z&&[^m-p]]    // "a" bis "z", außer "m" bis "p": `[a-lq-z]` (Subtraktion)
```

2.2.1.11 Handling of TeX Shenanigans

Zustand: (Formale) Beschreibung eines Zustandes der Welt

Aktion: (Formale) Beschreibung einer durch Agenten ausführbaren Aktion

- Anwendbar auf bestimmte Zustände
- Überführt Welt in neuen Zustand (“Nachfolge-Zustand”)

[1] sometime even more often
[2] lalalala

LaTeX-Befehle wie `\bigskip` etc. sollten automatisch entfernt werden:

Hier nach den LaTeX-Befehlen.

Geeignete Abstraktionen wählen für Zustände und Aktionen!

2.2.1.12 Pandoc Shenanigans

Pandoc sometimes transforms whitespace into UTF8 whitespace, which results in pdftex yelling at me, when this appears in lstlistings.

Example:

```
cf. https://www.overleaf.com/learn/latex/Beamer%23Creating_a_table_of_contents
```

normal white space: cf. https://www.overleaf.com/learn/latex/Beamer%23Creating_a_table_of_contents

pandoc replacement: cf. https://www.overleaf.com/learn/latex/Beamer%23Creating_a_table_of_contents

Also … hmmm. And “wuppie”?

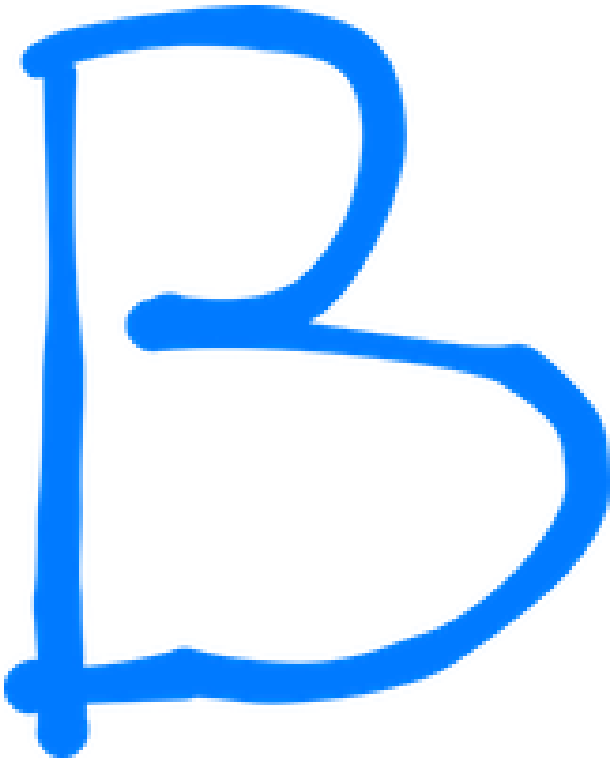
2.2.1.13 Columns

blablabla

Zum Parsen von Ausdrücken (*Expressions*) könnte man diese einfache Grammatik einsetzen. Ein Ausdruck ist dabei entweder ein einfacher Integer oder eine Addition oder Multiplikation zweier Ausdrücke.

```
expr : e1=expr '*' e2=expr      # MUL
     | e1=expr '+' e2=expr      # ADD
     | INT                      # NUM
     ;
```

Beim Parsen von “5*4+3” würde dabei der folgende Parsetree entstehen:



wuppie! fluppie! foo? bar …

2.2.1.14 Credits Typische Regeln und Konventionen tauchen überall auf, beispielsweise bei Tim Pope (siehe nächstes Beispiel) oder bei "How to Write a Git Commit Message".

```
Short (50 chars or less) summary of changes

More detailed explanatory text, if necessary.  Wrap it to about
72 characters or so.  In some contexts, the first line is treated
as the subject of an email and the rest of the text as the body.
The blank line separating the summary from the body is critical
(unless you omit the body entirely); tools like rebase can get
confused if you run the two together.

Further paragraphs come after blank lines.

 - Bullet points are okay, too
 - Typically a hyphen or asterisk is used for the bullet, preceded
   by a single space, with blank lines in between, but conventions
   vary here
```

Quelle: "A Note About Git Commit Messages" by Tim Pope on tbaggery.com

---

Use `[bla]{.credits nolist=true}` to put a nicely formatted reference to the original sources in the text without adding it to the list of exceptions to our licence (just giving credits):

Quelle: Test 1: Eigenes Material basierend auf einer Idee nach XYZ.

Note: Using the attribute “nolist” with any value would prevent this span from being included in the exceptions list since values will be read as string in the filter. So even `[bla]{.credits nolist=false}` will work:

Quelle: Test 2: Eigenes Material basierend auf einer Idee nach XYZ.

Quelle: Test 3: Eigenes Material basierend auf einer Idee nach XYZ.

Quelle: Test 4: Eigenes Material basierend auf einer Idee nach XYZ.

Quelle: Test 5: Eigenes Material basierend auf einer Idee nach XYZ.

Do not use the old `origin` span anymore - superceded by `credits`. This should emit a warning…

2.2.1.15 Filters

2.2.1.15.1 ShowMe

Hier ein ShowMe-Test:

this is hidden content …

(but not used anymore)

Use `details` instead:

this is hidden content …

wuppie

this is a show-me w/ title :)

2.2.1.15.2 CBOX

this is content to be centered (and put into a box)…

(but not used anymore)

2.2.1.15.3 Center

this content should be centered

2.2.1.15.4 Alert

This will be highlighted. (but not used anymore)

Use Pandoc’s mark extension instead: This will be highlighted. Even **with bold** text.

2.2.1.15.5 Hinweis

This is a hint. (but not used anymore)

2.2.1.15.6 Thema The topic of this task or … (but not used anymore)

2.2.1.15.7 BSP Lalalelu

Simple Beispiel-Button X (but not used anymore)

lalelu

use `ex` instead:

Simple Beispiel-Button X

Beispiel-Button w/ link

Vor `\pause`…

Nach `\pause`… (“neue” Slide)

2.2.1.16 Last Change should be added automatically and in `\scriptsize` or `<sup><sub>`

Zum Nachlesen

- Tate (2010, Kap. 2): foo bar wuppie fluppie
- Tate (2010, Kap. 2): Creating Graphical User Interfaces > Creating a GUI With Swing
- Nystrom (2021): Abschnitt 2.5.2: Ant
- (Nystrom 2021): Abschnitt 2.5.2: Ant

Lernziele

- k1: K1
- k2: K2
- k3: K3.1
- k3: K3.2

Quizzes

- Quiz Git Basics (ILIAS)

Challenges

Lexer und Parser mit ANTLR: Programmiersprache Lox

Betrachten Sie folgenden Code-Schnipsel in der Sprache "Lox":

```
fun fib(x) {
    if (x == 0) {
        return 0;
    } else {
        if (x == 1) {
            return 1;
        } else {
            fib(x - 1) + fib(x - 2);
        }
    }
}

var wuppie = fib(4);
```

Erstellen Sie für diese fiktive Sprache einen Lexer+Parser mit ANTLR. Implementieren Sie mit Hilfe des Parse-Trees und der Listener oder Visitoren einen einfachen Pretty-Printer.
(Die genauere Sprachdefinition finden Sie bei Bedarf unter craftinginterpreters.com/the-lox-language.html.)

Test for Pandoc Filter

This should appear only in GFM/Docsify/PDF, but NOT in Beamer (i.e. not in license statement!).

Quelle: test from yaml (challenges) - should not appear in slides

2.2.2 NN02 - Lineare Regression und Gradientenabstieg

Videos

- NN2.1 - Lineare Regression - Intro
- NN2.2 - Vergleich Perzeptron und Bsp
- NN2.3 - Kostenfunktiıon und Gradientenvektor
- NN2.4 - Berechnung Gradientenvektor - Beispiel
- NN2.5 - Berechnung Gradientenvektor - Allgemein
- NN2.6 - Skalierung der Merkmale

Weitere Unterlagen

- NN02-Lineare_Regression.pdf

2.2.2.1 Kurze Übersicht

2.2.2.1.1 Formalisierung (original)

- Ausgabe y ist reelle Zahl aus einem stetigen Bereich (zum Beispiel Hauspreis)

- Die **Hypothesenfunktion** ist eine gewichtete Summe der Merkmale x_i plus eine Konstante w_0:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- Der **Verlust** (engl. loss) für einen Datenpunkt $\mathbf{x}$ ist das **Fehlerquadrat**:

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Die Kosten (engl. cost) sind der durchschnittliche Verlust über alle Datenpunkte:

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2 = \frac{1}{2m}\sum_{i=1}^{m}(h(\mathbf{x}) - y)^2$$

2.2.2.1.2 Formalisierung (ohne inline math)

- Ausgabe y ist reelle Zahl aus einem stetigen Bereich (zum Beispiel Hauspreis)
- Die **Hypothesenfunktion** ist eine gewichtete Summe der Merkmale x_i plus eine Konstante w_0:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- Der **Verlust** (engl. loss) für einen Datenpunkt x ist das **Fehlerquadrat**:

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Die Kosten (engl. cost) sind der durchschnittliche Verlust über alle Datenpunkte:

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2 = \frac{1}{2m}\sum_{i=1}^{m}(h(\mathbf{x}) - y)^2$$

2.2.2.1.3 Formalisierung (mit inline math, block math ausgerückt)

- Ausgabe y ist reelle Zahl aus einem stetigen Bereich (zum Beispiel Hauspreis)
- Die **Hypothesenfunktion** ist eine gewichtete Summe der Merkmale x_i plus eine Konstante w_0:

$$h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

- Der **Verlust** (engl. loss) für einen Datenpunkt x ist das **Fehlerquadrat**:

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Die Kosten (engl. cost) sind der durchschnittliche Verlust über alle Datenpunkte:

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2 = \frac{1}{2m}\sum_{i=1}^{m}(h(\mathbf{x}) - y)^2$$

2.2.2.1.4 Der Gradient

- Der **Gradientenvektor** $\nabla J(\mathbf{w})$ setzt sich zusammen aus den partiellen Ableitungen der Kostenfunktion J nach den Gewichten w_i und zeigt in jedem Punkt $\mathbf{w}$ in die **Richtung des steilsten Aufstiegs**:

$$\nabla J = [\partial J/\partial w_0 \quad \partial J/\partial w_1 \quad \ldots \quad \partial J/\partial w_n]^T$$

- **Schlussfolgerung**: In die entgegengesetzte Richtung, i.e. in Richtung $-\nabla J(\mathbf{w})$ geht es am *steilsten bergab!*
- **IDEE**: Bewege $\mathbf{w}$ in Richtung $-\nabla J(\mathbf{w})$, um die Kosten J möglichst schnell zu senken.

2.2.2.1.5 Der Gradientenabstieg (engl. Gradient Descent)

1. Starte mit zufälligen Gewichten $\mathbf{w}$
2. Berechne den Gradientenvektor im aktuellen Punkt $\mathbf{w}$
3. **Gewichtsaktualisierung**: Gehe einen *kleinen* Schritt in Richtung $-\nabla J(\mathbf{w})$

$$\mathbf{w}_{neu} := \mathbf{w}_{alt} - \alpha \cdot \nabla J(\mathbf{w}_{alt})$$

   (α: Lernrate/Schrittweite).
4. Wiederhole Schritte 2-3, bis das globale Minimum von J erreicht ist.

Lernziele

- k2: Lineare Regression aus Sicht neuronaler Netze: Graphische Darstellung, Vergleich mit Perzeptron
- k2: Formalisierung
- k2: Verlust- und Kostenfunktion
- k2: Gradientenvektor
- k2: Lernrate
- k3: Gradientenabstieg

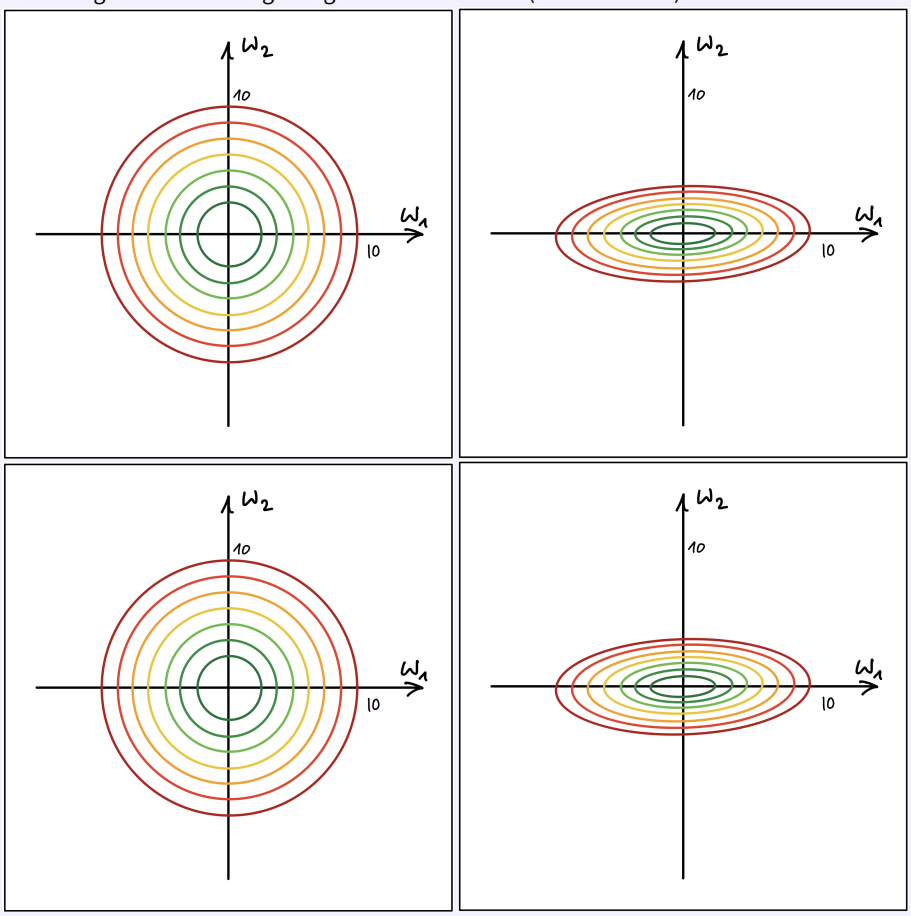
Quizzes

- Selbsttest Lineare Regression (ILIAS)

Challenges

Skalierung der Merkmale

Abbildung 1 und Abbildung 2 zeigen die Höhenlinien (Contour Lines) von zwei Kostenfunktionen.



- Erklären Sie, welcher der beiden Fälle nachteilhaft für den Gradientenabstieg Algorithmus ist. Wo liegt der Nachteil? Wie kann die Merkmalskalierung dem genannten Nachteil entgegenwirken?
- Zeigen Sie unter Verwendung Ihrer eigenen, zufällig generierten Datenpunkte aus dem Bereich $[100, 300] \times [0, 2]$, wie sich Standardisierung, Min-Max Skalierung und Normalisierung auf die Daten auswirken. Vergleichen Sie dazu die jeweiligen Streudiagramme (scatterplots). Sie können hierzu das folgende **Jupyter Notebook** als Startpunkt benutzen.

2.2.3 subfolder

Weitere Unterlagen

- NN1-Das_Perzeptron.pdf
- Annotierte Folien: LR-Parser1
- Annotierte Folien: LR-Parser1 (RAW)

2.2.3.1 Single page 'Foo' in a leaf bundle

2.2.3.1.1 Lecture/03-Test/Subfolder/Foo.md This is a “single page” in a “leaf bundle” and should **not** be available.

3 Praktikum

Hier finden Sie die Übungsblätter.

3.1 Blatt 04: Semantische Analyse

3.1.1 Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung einer Symboltabelle und eines einfachen Type-Checkers für eine fiktive statisch typisierte Sprache mit Expressions, Kontrollstrukturen und Funktionen.

3.1.2 Methodik

Sie finden im Sample Project eine Grammatik, die (teilweise) zu der Zielsprache auf diesem Blatt passt. Analysieren Sie diese Grammatik und vervollständigen Sie diese bzw. passen Sie diese an.

Erstellen Sie mit dieser Grammatik und ANTLR wieder einen Lexer und Parser. Definieren Sie einen AST und konvertieren Sie Ihren Parse-Tree in einen AST.

Es ist empfehlenswert, den Type-Checker dreiphasig zu realisieren:

1. Aufbauen der Symboltabelle und Prüfen von z.B. Deklaration/Definition vs. Benutzung (Variablen) usw.
2. Prüfen bei Funktionsaufrufen auf vorhandene/sichtbare Funktionsdefinitionen
3. Prüfen der verwendeten Typen

3.1.3 Sprachdefinition

Ein Programm besteht aus einer oder mehreren Anweisungen (*Statements*).

3.1.3.1 Anweisungen (*Statements*) Eine Anweisung ist eine Befehlsfolge, beispielsweise eine Deklaration (Funktionen), Definition (Variablen, Funktionen), Zuweisung, ein Funktionsaufruf oder eine Operation. Sie muss immer mit einem Semikolon abgeschlossen werden. Eine Anweisung hat keinen Wert.

```
int a;           # Definition der Integer-Variablen a
int b = 10 - 5;  # Definition der Integer-Variablen b und Zuweisung des Ausdruckes 10-5 (Integer-Wert 5)
c = "foo";       # Zuweisung des Ausdrucks "foo" (String) an die Variable c (diese muss dazu vorher definiert und sichtbar sein)
func1(a, c);     # Funktionsaufruf mit Variablen a und c
```

Kontrollstrukturen und Code-Blöcke sowie `return`-Statements zählen ebenfalls als Anweisung.

3.1.3.2 Code-Blöcke und Scopes Code-Blöcke werden in geschweifte Klammern eingeschlossen und enthalten eine beliebige Anzahl von Anweisungen.

Jeder Code-Block bildet einen eigenen Scope - alle Deklarationen/Definition in diesem Scope sind im äußeren Scope nicht sichtbar. Im Scope kann auf die Symbole des/der umgebenden Scopes zugegriffen werden. Symbole in einem Scope können gleichnamige Symbole im äußeren Scope verdecken.

```
# globaler Scope
int a = 7;
int d;
{   # erster innerer Scope
    int b = 7;      # b ist nur in diesem und im zweiten inneren Scope sichtbar
    foo(a);         # Funktionsaufruf mit der Variable a aus dem äußeren Scope (Wert 7)
    int a = 42;     # Variable verdeckt das a aus dem äußeren Scope
    foo(a);         # Funktionsaufruf mit der Variable a aus dem aktuellen Scope (Wert 42)
    {   # zweiter innerer Scope
        int c = 9;  # dieses c ist nur hier sichtbar
        foo(d);     # d wird im äußeren Scope gesucht, dort nicht gefunden und im nächsthöheren Scope gesucht (rekursiv)
    }
}
{   # dritter innerer Scope
    int b;          # dieses b hat mit dem b aus dem ersten inneren Scope nichts zu tun
    foo(a);         # Funktionsaufruf mit der Variable a aus dem äußeren Scope (Wert 7)
}
```

3.1.3.3 Ausdrücke (*Expressions*) Die einfachsten Ausdrücke sind Integer- oder String-Literale. Variablen und Funktionsaufrufe sind ebenfalls Ausdrücke. Komplexere Ausdrücke werden mit Hilfe von Operationen gebildet, dabei sind die Operanden jeweils auch wieder Ausdrücke.

Ein Ausdruck hat immer einen Wert und einen Typ.

Die Operatoren besitzen eine Rangfolge, um verschachtelte Operationen aufzulösen. Sie dürfen daher nicht einfach von links nach rechts aufgelöst werden. Die Rangfolge der Operatoren entspricht der üblichen Semantik (vgl. Java, C, Python).

Es gibt in unserer Sprache folgende Operationen mit der üblichen Semantik:

3.1.3.3.1 Vergleichsoperatoren

Operation	Operator
Gleichheit	==
Ungleichheit	!=
Größer	>

Operation	Operator
Kleiner	<

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `string` und **`int`** zulässig. Das Ergebnis ist immer vom Typ `bool`.

3.1.3.3.2 Arithmetische Operatoren

Operation	Operator
Addition / String-Literal-Verkettung	+
Subtraktion	−
Multiplikation	*
Division	/

Die Operanden müssen jeweils beide den selben Typ haben. Für + sind `string` und **`int`** zulässig, für die anderen Operatoren (−, *, /) nur **`int`**. Das Ergebnis ist vom Typ der Operanden.

3.1.3.3.3 Beispiele für Ausdrücke

```
10 - 5        # Der Integer-Wert 5
"foo"         # Der String "foo"
a             # Wert der Variablen a (Zugriff auf a)
a + b         # Ergebnis der Addition der Variablen a und b
func1(a, b)   # Ergebnis des Funktionsaufrufs
```

Ausdrücke werden nicht mit einem Semikolon abgeschlossen. Sie sind also Teil einer Anweisung.

3.1.3.4 Bezeichner Werden zur Bezeichnung von Variablen und Funktionsnamen verwendet. Sie bestehen aus einer Zeichenkette der Zeichen `a-z`,`A-Z`, `0-9`. Bezeichner dürfen nicht mit einer Ziffer `0-9` beginnen.

3.1.3.5 Variablen Variablen bestehen aus einem eindeutigen Bezeichner (Variablennamen). Den Variablen können Werte zugewiesen werden und Variablen können als Werte verwendet werden. Die Zuweisung erfolgt mithilfe des =-Operators. Auf der rechten Seite der Zuweisung können auch Ausdrücke stehen.

```
int a;          # Definition der Variablen a (Typ: Integer)
int a = 7;      # Definition und Initialisierung einer Variablen
a = 5;          # Zuweisung des Wertes 5 an die Variable a
a = 2 + 3;      # Zuweisung des Wertes 5 an die Variable a
```

Variablen müssen vor ihrer Benutzung (Zugriff, Zuweisung) definiert und im aktuellen Scope sichtbar sein. Die Initialisierung kann zusammen mit der Definition erfolgen.

Variablen können in einem Scope nicht mehrfach definiert werden.

```
int a = 42;

{
    a = 7;  # zulässig - a ist hier sichtbar, Zugriff auf globalen Scope
    b = 9;  # unzulässig - b ist nicht definiert

    int a;  # zulässig - erste Definition in diesem Scope
    int a;  # unzulässig - a ist in diesem Scope bereits definiert
}
```

3.1.3.6 Kommentare Kommentare werden durch das Zeichen # eingeleitet und umfassen sämtliche Zeichen bis zum nächsten Newline.

3.1.3.7 Kontrollstrukturen

3.1.3.7.1 While-Schleife While-Schleifen werden mit dem Schlüsselwort `while` eingeleitet. Sie bestehen im Weiteren aus einer Bedingung in runden Klammern und einem in geschweiften Klammern formulierten Code-Block.

Die Bedingung besteht aus einem Vergleichsausdruck.

```
while (<Bedingung>) {
    <Anweisung_1>
    <Anweisung_2>
}
```

```
int a = 10;

while (a > 0) {
    a = a - 1;
}
```

3.1.3.7.2 Bedingte Anweisung (If-Else) Eine bedingte Anweisung besteht immer aus genau einer `if`-Anweisung, und einer oder keiner `else`-Anweisung.

Eine `if`-Anweisung wird mit dem Schlüsselwort `if` eingeleitet und besteht aus einer Bedingung in runden Klammern und einem in geschweiften Klammern formulierten Code-Block.

Die Bedingung besteht aus einem Vergleichsausdruck.

Eine `else`-Anweisung wird mit dem Schlüsselwort `else` eingeleitet. Auf das Schlüsselwort folgt in geschweiften Klammern formulierter Code-Block.

```
if (<Bedingung>) {
    <Anweisung_1>
    <Anweisung_2>
}
```

```
if (<Bedingung>) {
    <Anweisung>
} else {
    <Anweisung>
}
```

```
int a = 42;

if (a > 0) {
    a = a - 1;
    if (a > 0) {
        a = a - 1;
    }
} else {
    a = a + 1;
}
```

3.1.3.8 Funktionen

3.1.3.8.1 Funktionsdefinition Eine Funktionsdefinition macht dem Compiler die Implementierung einer Funktion bekannt.

Sie gibt zunächst die Signatur der Funktion (den "Funktionskopf") bekannt: Rückgabetyp, Funktionsname, Parameterliste.

Die Parameterliste ist eine Komma-separierte Liste mit der Deklaration der Parameter (jeweils Typ und Variablenname). Die Parameterliste kann auch leer sein.

Nach dem Funktionskopf folgt der Körper der Funktion als Code-Block.

Funktionen können in einem Scope nicht mehrfach definiert werden.

```
type bezeichner(type param1, type param2) {
    <Anweisung_1>
    <Anweisung_2>
    return <Bezeichner, Wert oder Operation>;
}
```

```
bool func1(int a, string b) {
    int c = a + f2(b);
    return c == 42;
}
```

3.1.3.8.2 Funktionsaufrufe Funktionsaufrufe bestehen aus einem Bezeichner (Funktionsname) gefolgt von einer in Klammern angegebenen Liste der Argumente, die auch leer sein kann. Als Argumente können alle passend typisierten Ausdrücke dienen.

```
func1(5, var1)
func1(func2(), 1 + 1)
```

Die aufgerufene Funktion muss im aktuellen Scope sichtbar sein, der Funktionsaufruf muss zur Definition passen.

Die aufgerufene Funktion muss (im Gegensatz zum Zugriff auf Variablen) nicht vor dem ersten Aufruf definiert sein. Folgender Code ist also zulässig:

```
int a = 42;
if (a == 42) {
    foo(5);    # zulässig: foo ist erst nach diesem Aufruf definiert, aber in diesem Scope sichtbar
}

int foo(int a) {
    return a + 37;
}
```

3.1.3.9 Datentypen

Unsere Sprache hat drei eingebaute Datentypen:

Datentyp	Definition der Literale
`int`	eine beliebige Folge der Ziffern `0-9`
`string`	eine beliebige Folge von ASCII-Zeichen, eingeschlossen in `"`
`bool`	eines der beiden Schlüsselwörter `T` oder `F`

3.1.3.10 Beispiele

```
string a = "wuppie fluppie";
```

```
int a = 0;
if (10 < 1) {
    a = 42;
} else {
    foo();
}
```

```
int f95(int n) {
    if (n == 0) {
        return 1;
    } else {
        if (n == 1) {
            return 1;
        } else {
            return f95(n - 1) + f95(n - 2) + f95(n - 3) + f95(n - 4) + f95(n - 5);
        }
    }
}

int n = 10;
f95(n);
```

3.1.4 Aufgaben

3.1.4.1 Grammatik und AST (2P) Erstellen Sie eine ANTLR-Grammatik für die Zielsprache. Sie können dabei die Grammatik im Sample Project als Ausgangspunkt nutzen und diese anpassen und vervollständigen.

Definieren Sie einen AST für die Zielsprache. Welche Informationen aus dem Eingabeprogramm müssen repräsentiert werden?

Programmieren Sie eine Traversierung des Parse-Trees, die den AST erzeugt. Testen Sie dies mit den obigen Beispielprogrammen und definieren Sie sich selbst weitere Programme unterschiedlicher Komplexität für diesen Zweck.

3.1.4.2 Aufbau der Symboltabelle (2P) Bauen Sie für den AST eine Symboltabelle auf. Führen Sie dabei die im ersten Lauf möglichen Prüfungen durch, beispielsweise ob referenzierte Variablen tatsächlich bereits definiert und sichtbar sind oder ob eine Variable oder Funktion in einem Scope mehrfach definiert wird oder ob Variablen als Funktion genutzt werden. Geben Sie erkannte Fehler auf der Konsole aus.

3.1.4.3 Symboltabelle: Funktionsaufrufe (1P) Implementieren Sie einen zweiten Lauf. Dabei soll für Funktionsaufrufe geprüft werden, ob diese Funktionen bereits definiert sind und im Scope sichtbar sind. Geben Sie erkannte Fehler auf der Konsole aus.

3.1.4.4 Symboltabelle: Typprüfungen (5P) Implementieren Sie einen dritten Lauf. Führen Sie die Typprüfung durch: Haben die Operanden in Ausdrücken die richtigen Typen, passen die Typen der Funktionsargumente, passen die Typen bei einer Zuweisung, … Geben Sie erkannte Fehler auf der Konsole aus. *Hinweis*: Sie brauchen hier nur die Typprüfung durchführen. Eine Typinferenz oder Typerweiterung o.ä. ist nicht notwendig.

3.2 Blatt 01: Reguläre Sprachen

3.2.1 Sprachen von regulären Ausdrücken (1P)

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$a + a\,(a + b)^*\,a$

3.2.2 Bezeichner in Programmiersprachen (3P)

Betrachten Sie eine Programmiersprache, in der die Bezeichner (= Namen für Variablen, Funktionen, Klassen, Methoden, …) folgenden Aufbau haben:

- Alle Variablennamen beginnen mit **V** oder **v**
- Handelt es sich um globale Variablen, beginnen Sie mit **V**, lokale beginnen mit **v**
- Funktions- und Methodenparameter beginnen mit **p**, KLassenparameter (bei der Definition von Vererbung) beginnen mit **P**

- Weitere Bezeichner müssen mit einem Buchstaben (a-z, A-Z) beginnen
- Die folgenden Zeichen dürfen Buchstaben, Ziffern und ein Untersreich sein
- Bezeichner dürfen nicht mit einem Unterstrich enden
- Alle Bezeichner müssen aus mindestens zwei Zeichen bestehen

Entwickeln Sie einen regulären Ausdruck, der den Aufbau der Bezeichner beschreibt. Beachten Sie, dass Ihr regex alle zulässigen Bezeichner beschreiben muss, aber keinen einzigen unzulässigen beschreiben darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom regex gematcht werden.

Entwickeln Sie einen DFA, der diese Bezeichner akzeptiert. Beachten Sie, dass Ihr DFA alle zulässigen Bezeichner akzeptieren muss, aber keinen einzigen unzulässigen akzeptieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom Automaten zeichenweise gelesen und akzeptiert werden.

Entwickeln Sie eine reguläre Grammatik, die diese Bezeichner generiert. Beachten Sie, dass Ihre Grammatik alle zulässigen Bezeichner generieren können muss, aber keinen einzigen unzulässigen generieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie die Ableitungsbäume dazu.

3.2.3 Gleitkommazahlen in Programmiersprachen (2P)

Recherchieren Sie zunächst den Aufbau von Gleitkommazahlen in Python und Java.

Erstellen Sie für jede der beiden Programmiersprachen reguläre Ausdrücke, DFAs und reguläre Grammatiken wie in Aufgabe 1. Verifizieren Sie Ihre Lösungen wie in Aufgabe

1.

3.2.4 Mailadressen? (1P)

Warum ist der folgende regex ungeeignet für die Verarbeitung von Mailadressen?

$(a-z)^{+}@(a-z).(a-z)$

Bitte beachten Sie, dass die Schreibweise a-z nicht unserer Definition genügt. Eigentlich müsste jedes Zeichen aufgeführt werden:

$a+b+c+c+\ldots+z$ ist besser, aber immer noch nicht richtig. Warum?

Anmerkung: Diese Darstellung wird ab jetzt akzeptiert.

Verbessern Sie den gegebenen regulären Ausdruck.

3.2.5 Der zweitletzte Buchstabe (1P)

Entwickeln Sie einen DFA, der nur Wörter über $\Sigma = \{1, 2, 3\}$ akzeptiert, deren zweitletztes Zeichen dasselbe ist wie das zweite.

3.2.6 Sprache einer regulären Grammatik (2P)

Welche Sprache generiert die folgende Grammatik?

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow dB \mid bA \mid cA \\ B &\rightarrow ac \mid bC \mid cA \\ C &\rightarrow \epsilon \end{aligned}$$

3.3 Blatt 02: CFG

3.3.1 PDA (2P)

Erstellen Sie einen deterministischen PDA, der die Sprache

$$L = \{w \in \{a, b, c\}^* \mid w \text{ hat doppelt so viele a's wie c's}\}$$

akzeptiert.

Beschreiben Sie Schritt für Schritt, wie der PDA die Eingaben *bcaba* und *bccac* abarbeitet.

3.3.2 Akzeptierte Sprache (1P)

Ist der folgenden PDA deterministisch? Warum (nicht)?

q_4 sei der akzeptierende Zustand.

$$\begin{aligned} \delta(q_0, a, \perp) &= (q_0, A \perp) \\ \delta(q_0, a, A) &= (q_0, AA) \\ \delta(q_0, b, A) &= (q_1, BA) \\ \delta(q_1, b, B) &= (q_1, BB) \\ \delta(q_1, c, B) &= (q_2, \epsilon) \\ \delta(q_2, c, B) &= (q_2, \epsilon) \\ \delta(q_2, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, AA) \\ \delta(q_3, \epsilon, \perp) &= (q_4, \epsilon) \end{aligned}$$

Zeichnen Sie den Automaten. Geben Sie das 7-Tupel des PDa an. Welche Sprache akzeptiert er?

3.3.3 Kontextfreie Sprache (1P)

Welche Sprache generiert die folgende kontextfreie (Teil-) Grammatik?

$$G = (\{\text{Statement}, \text{Condition}, \ldots\}, \{\text{"if"}, \text{"else"}, \ldots\}, P, \text{Statement})$$

mit

$$\begin{aligned} P = \{ & \\ & \text{Statement} \rightarrow \text{"if" Condition Statement} \mid \text{"if" Condition Statement "else" Statement} \\ & \text{Condition} \rightarrow \ldots \\ \} & \end{aligned}$$

Ist die Grammatik mehrdeutig? Warum (nicht)?

3.3.4 Kontextfreie Grammatik (2P)

Entwickeln Sie eine kontextfreie Grammatik für die Sprache

$$L = \{a^i b^j c^k \mid i = j \vee j = k\}$$

Zeigen Sie, dass die Grammatik mehrdeutig ist. Entwickeln Sie einen PDA für diese Sprache.

3.3.5 Kontextfreie Grammatik (4P)

Betrachten sie die folgende Grammatik:

$$G = (\{S, A\}, \{1, 2, 3\}, P, S)$$

mit

$$\begin{aligned} P = \{ & \\ & S \rightarrow 1AS \mid 3 \\ & A \rightarrow 2AS \mid \epsilon \\ \} & \end{aligned}$$

Berechnen die die *First-* und *Follow-Mengen* der Grammatik.

Zeigen Sie, dass die Grammatik LL(1) ist.

Konstruieren Sie die LL-Parsertabelle für die Grammatik und simulieren Sie das Parsen des Wortes *1233*.

3.4 Blatt 03: ANTLR

3.4.1 Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung eines einfachen *Pretty Printers* für eine einfache fiktive Sprache mit Expressions und Kontrollstrukturen.

Dazu werden Sie eine passende kontextfreie Grammatik definieren mit Lexer- und Parser-Regeln und dabei auch übliche Vorrangregeln beachten.

Für diese Grammatik erstellen Sie mit Hilfe von ANTLR einen Lexer und einen Parser, die zu einem Eingabeprogramm einen Parse-Tree erzeugen.

Den im Parse-Tree repräsentierten Code des Eingabeprogramms können Sie mit Hilfe einer Traversierung konsistent eingerückt wieder auf der Standardausgabe ausgeben - das ist der *Pretty Printer*.

Sie werden merken, dass viele Strukturen im Parse-Tree für diese Aufgabe nicht relevant sind und den Baum mit einer weiteren Traversierung in einen vereinfachten Baum, den sogenannten Abstract-Syntex-Tree (*AST*), transformieren und diesen erneut als formatierten Code auf der Konsole ausgeben.

3.4.2 Methodik

Nutzen Sie das Starter-Projekt in der Vorgabe.

Laden Sie sich das Projekt herunter, binden Sie es in Ihre IDE ein und vergewissern Sie sich, dass alles funktioniert: Führen Sie das enthaltene Programm aus, ändern Sie die mitgelieferten Beispielgrammatik.

Bauen Sie dann Ihre Grammatik für die Aufgabe schrittweise auf. Testen Sie diese mit Hilfe der Beispielprogramme der Zielsprache (s.u.) und überlegen Sie sich selbst weitere Code-Schnipsel, die Sie mit Ihrem Parser einlesen bzw. die Ihr Parser zurückweisen sollte.[3] Es empfiehlt sich, in dieser Phase mit dem ANTLR-Plugin für IntelliJ zu arbeiten.

Erkunden Sie dann die Strukturen Ihres Parse-Trees. Diese sind an die Regeln Ihrer Grammatik gekoppelt und sind deshalb so individuell wie Ihre Grammatik. Mit einer Traversierung des Baumes können Sie die gewünschte Ausgabe programmieren und auch die Erstellung des vereinfachten Baumes (AST).

3.4.3 Sprachdefinition

Ein Programm besteht aus einer oder mehreren Anweisungen (*Statements*).

[3] Um den Text lesbar zu halten, wird hier oft nur von “Parser” gesprochen - gemeint ist aber die gesamte auf diesem Blatt zu erstellende Toolchain: Lexer - Parser - AST - Ausgabe.

3.4.3.1 Anweisungen (*Statements*) Eine Anweisung ist eine einzeilige Befehlsfolge, beispielsweise eine Zuweisung oder eine Operation. Sie muss immer mit einem Newline abgeschlossen werden. Eine Anweisung hat keinen Wert.

```
a := 10 - 5  # Zuweisung des Ausdruckes 10-5 (Integer-Wert 5) an die Variable a
b := "foo"   # Zuweisung des Ausdrucks "foo" (String) an die Variable b
```

Kontrollstrukturen (s.u.) zählen ebenfalls als Anweisungen.

3.4.3.2 Ausdrücke (*Expressions*) Die einfachsten Ausdrücke sind Integer- oder String-Literale. Variablen sind ebenfalls Ausdrücke. Komplexere Ausdrücke werden mit Hilfe von Operationen gebildet, dabei sind die Operanden jeweils auch wieder Ausdrücke. Ein Ausdruck hat/ergibt immer einen Wert.

Die Operatoren besitzen eine Rangfolge, um verschachtelte Operationen aufzulösen. Sie dürfen daher nicht einfach von links nach rechts aufgelöst werden. Die Rangfolge der Operatoren entspricht der üblichen Semantik (vgl. Java, C, Python).

Es gibt in unserer Sprache folgende Operationen mit der üblichen Semantik:

3.4.3.2.1 Vergleichsoperatoren

Operation	Operator
Gleichheit	==
Ungleichheit	!=
Größer	>
Kleiner	<

3.4.3.2.2 Arithmetische Operatoren

Operation	Operator
Addition / String-Literal-Verkettung	+
Subtraktion	−
Multiplikation	*
Division	/

3.4.3.2.3 Beispiele für Ausdrücke

```
10 - 5   # Der Integer-Wert 5
"foo"    # Der String "foo"
a        # Wert der Variablen a
a + b    # Ergebnis der Addition der Variablen a und b
```

3.4.3.3 Bezeichner Werden zur Bezeichnung von Variablen verwendet. Sie bestehen aus einer Zeichenkette der Zeichen `a-z`,`A-Z`, `0-9`, `_`. Bezeichner dürfen nicht mit einer Ziffer `0-9` beginnen.

3.4.3.4 Variablen Variablen bestehen aus einem eindeutigen Bezeichner (Variablennamen). Den Variablen können Werte zugewiesen werden und Variablen können als Werte verwendet werden. Die Zuweisung erfolgt mithilfe des `:=`-Operators. Auf der rechten Seite der Zuweisung können auch Ausdrücke stehen.

```
a := 5      # Zuweisung des Wertes 5 an die Variable a
a := 2 + 3  # Zuweisung des Wertes 5 an die Variable a
```

3.4.3.5 Kommentare Kommentare werden durch das Zeichen # eingeleitet und umfassen sämtliche Zeichen bis zum nächsten Newline.

3.4.3.6 Kontrollstrukturen

3.4.3.6.1 While-Schleife While-Schleifen werden mit dem Schlüsselwort **`while`** eingeleitet. Sie bestehen im Weiteren aus einer Bedingung, die durch ein **`do`** abgeschlossen wird, einer Folge von Anweisungen und werden mit dem Schlüsselwort `end` abgeschlossen.

Die Bedingung kann aus einem Vergleichsausdruck bestehen.

```
while <Bedingung> do
    <Anweisung_1>
    <Anweisung_2>
end
```

```
a := 10
b := 0
while a >= 0 do
    a := a - 1
    b := b + 9
end
```

3.4.3.6.2 Bedingte Anweisung (If-Else) Eine bedingte Anweisung besteht immer aus genau einer **`if`**-Anweisung, gefolgt von einer Bedingung, die mit einem **`do`** abgeschlossen wird und einer Folge von Anweisungen.

Danach wird die bedingte Anweisung entweder mit dem Schlüsselwort `end` abgeschlossen oder es folgt genau ein **`else`**-Teil.

Ein **`else`**-Teil wird mit dem Schlüsselwort **`else`** eingeleitet. Darauf folgt ein **`do`** und eine Folge von Anweisungen. Der **`else`**-Teil wird mit dem Schlüsselwort `end` abgeschlossen.

```
if <Bedingung> do
    <Anweisung_1>
    <Anweisung_2>
end
```

```
if <Bedingung> do
    <Anweisung>
else do
    <Anweisung>
end
```

```
a := "abc"
if a < "abc" do
    a := "wuppie"
else do
    a := "nope"
end
```

3.4.3.7 Datentypen Unsere Sprache hat zwei eingebaute Datentypen, für die entsprechende Literale erkannt werden müssen:

Datentyp	Definition der Literale
`int`	eine beliebige Folge der Ziffern `0-9`
`string`	eine beliebige Folge von ASCII-Zeichen, eingeschlossen in `"`

Die Sprache ist dynamisch typisiert, d.h. beim Parsen werden Ihnen keine Typ-Angaben im Code begegnen. Aber Sie müssen die entsprechenden Werte (Literale) parsen können.

3.4.3.8 Beispiele

```
a := "wuppie fluppie"
```

```
a := 0
if 10 < 1 do
    a := 42
else do
    a := 7
end
```

3.4.4 Aufgaben

3.4.4.1 Grammatik (4P) Definieren Sie für die obige Sprache eine geeignete ANTLR-Grammatik.

Sie werden sowohl Lexer- als auch (rekursive) Parser-Regeln benötigen. Beachten Sie die üblichen Vorrangregeln für die Operatoren, orientieren Sie sich hier an Sprachen wie Java oder Python oder C.

3.4.4.2 Pretty Printer (3P) Erzeugen Sie mithilfe der Grammatik und ANTLR einen Lexer und Parser. Damit können Sie syntaktisch korrekte Eingabe-Programme in einen Parse-Tree überführen.

Programmieren Sie eine Traversierung Ihres Parse-Trees, in der Sie syntaktisch korrekte Programme konsistent eingerückt ausgeben können.

Jede Anweisung soll auf einer eigenen Zeile stehen. Die Einrückung soll mit Leerzeichen erfolgen und konsistent sein. Sie brauchen keine Begrenzung der Zeilenlänge implementieren.

Demonstrieren Sie die Fähigkeiten an mehreren Beispielen mit unterschiedlicher Komplexität.

Beispiel:

Aus

```
a     := 0
    if    10 < 1
       do
a    :=     42      # Zuweisung des Wertes 42 an die Variable a
else do
        a :=      7
  end
```

soll

```
a := 0
if 10 < 1 do
    a := 42
else do
    a := 7
end
```

werden.

Hinweis: Es geht nur um die Ausgabe syntaktisch korrekter Programme. Sie brauchen sich um die Semantik (z.B. passende Typen wie etwa keine Multiplikation von Strings mit Integern o.ä.) noch keine Gedanken machen! Achten Sie auf die korrekten Einrücktiefen. Die Zeilenlänge spielt hier keine Rolle, es wird einfach direkt nach jedem Statement umgebrochen (bzw. wie bei den Kontrollstrukturen gezeigt).

Hinweis: Das Thema Pretty Printing ist interessant und kann recht schnell ziemlich aufwändig werden. Sie finden im Paper "A prettier printer" von Philip Wadler (Wadler 2003) und im Blog "The Hardest Program I've Ever Written" von Bob Nystrom (Nystrom 2015) gut geschriebene Beiträge, um tiefer in die Materie einzusteigen.

3.4.4.3 AST (3P) Beim Parsen bekommen Sie von ANTLR einen Parse-Tree zurück, der direkt die Struktur Ihrer Grammatik widerspiegelt. Die einzelnen Zweige sind damit in der Regel aber auch viel zu tief verschachtelt.

Überlegen Sie sich, welche Informationen/Knoten Sie für die formatierte Ausgabe wirklich benötigen - das ist Ihr Abstract-Syntex-Tree (*AST*).

Programmieren Sie eine Transformation des Parse-Tree in die von Ihnen hier formulierten AST-Strukturen. Dies können Sie beispielsweise mit einer passenden Traversierung (Visitor-Pattern) erreichen.

Passen Sie den Pretty-Printer so an, dass er auch den AST ausgeben kann. (Alternativ können auch einen zweiten Pretty-Printer für den AST implementieren.)

Quellen

Abelson, H., G. J. Sussmann, und J. Sussmann. 1996. *Structure and Interpretation of Computer Programs*. MIT Press. https://mitpress.mit.edu/sites/default/files/sicp/index.html.

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/.

Hopcroft, J. E., R. Motwani, und J. D. Ullman. 2003. *Einführung in die Automatentheorie, formale Sprachen und Komplexitätstheorie*. I theoretische informatik. Pearson Education Deutschland GmbH.

Nystrom, R. 2015. „The Hardest Program I've Ever Written". https://journal.stuffwithstuff.com/2015/09/08/the-hardest-program-ive-ever-written/.

Nystrom, R. 2021. *Crafting Interpreters*. Genever Benning. https://github.com/munificent/craftinginterpreters.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/.

Siek, J. G. 2023. *Essentials of Compilation: An Incremental Approach in Racket*. The MIT Press. https://github.com/IUCompilerCourse/Essentials-of-Compilation.

Tate, B. A. 2010. *Seven Languages in Seven Weeks*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/seven-languages-in/9781680500059/.

Wadler, P. 2003. „A Prettier Printer". *The Fun of Programming, Cornerstones of Computing*, 223–43. https://homepages.inf.ed.ac.uk/wadler/papers/prettier/prettier.pdf.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- “FooFOOOO” by me on void.intern.com
- test from yaml (challenges) - should not appear in slides
- “A Note About Git Commit Messages” by Tim Pope on tbaggery.com
- “Foo” by me on void.extern.com
- Turing Test version 3.png by Bilby on Wikimedia Commons (Public Domain)

> **Last modified:** 0f0b032 Sat Mar 21 19:10:03 2026 +0100 test (fresh build ...)